

Complete Interview Prep Master Guide

Generated automatically on: 2026-07-30

Topic: LeetCode

Code Solutions

AddBinary.java

```
package old.LeetCode;

import org.testng.Assert;

public class AddBinary {
    public static void main(String[] args) {

        Assert.assertEquals(addBinary("11","1"),"100");
    }

    public static String addBinary(String a, String b) {
        StringBuilder result = new StringBuilder();
        int carry = 0;
        int i = a.length() - 1;
        int j = b.length() - 1;

        while (i >= 0 || j >= 0 || carry > 0) {
            int sum = carry;
            if (i >= 0) {
                sum += a.charAt(i--) - '0';
            }
            if (j >= 0) {
                sum += b.charAt(j--) - '0';
            }

            result.append(sum % 2);
            carry = sum / 2;
        }

        return result.reverse().toString();
    }
}
```

ClimbingStairs.java

```
package old.LeetCode;

public class ClimbingStairs {
    //You are climbing a staircase. It takes n steps to reach the top.
    //Each time you can either climb 1 or 2 steps. In how many distinct ways can
    you climb to the top?

    public static void main(String[] args) {
        System.out.println(climbStairs(5));
    }

    public static int climbStairs(int n) {
        if (n <= 2) {
            return n;
        }
        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
}
```

ContainsDuplicate_1.java

```
package old.LeetCode;

import org.testng.Assert;

import java.util.HashSet;
import java.util.Set;

public class ContainsDuplicate_1 {

    //Given an integer array nums, return true if any value appears at least twice in
    the array, and return false if every element is distinct.

    /**
     * Time Complexity: O(n)
     * Space Complexity: O(n)
     */

    public static void main(String[] args) {
        Assert.assertTrue(containsDuplicate(new int[]{1, 2, 3, 1}));
        Assert.assertFalse(containsDuplicate(new int[]{1,2,3,4}));
    }
}
```

```

    public static boolean containsDuplicate(int[] nums) {
        Set<Integer> dupeFilter = new HashSet<>();
        for (int i : nums) {
            if (!dupeFilter.add(i)) {
                return true;
            }
        }
        return false;
    }
}

```

FirstOccurence.java

```

package old.LeetCode;

public class FirstOccurence {
    //Given two strings needle and haystack, return the index of the first
    //occurrence of needle in haystack, or -1 if needle is not part of haystack.

    public static void main(String[] args) {
        System.out.println(strStr("sadbutsad", "sad")==0);
    }

    public static int strStr(String haystack, String needle) {
        int m=haystack.length(),n=needle.length();
        for(int i=0;i<=m-n;i++){
            if(haystack.substring(i,i+n).equals(needle)){
                return i;
            }
        }
        return -1;
    }
}

```

GroupAnagrams_4.java

```

package old.LeetCode;

import java.util.*;

public class GroupAnagrams_4 {

    //Given an array of strings strs, group the anagrams together. You can return
    //the answer in any order.

    public static void main(String[] args) {
        System.out.println(groupAnagrams(new String[]{"eat", "tea", "tan", "ate",

```

```

"nat", "bat"}));
    System.out.println(groupAnagramsOptimised(new String[]{"eat", "tea",
"tan", "ate", "nat", "bat"}));
}

/**
 * Time Complexity: O(nlogn)
 * Space Complexity: O(n)
 */

public static List<List<String>> groupAnagrams(String[] strs) {
    List<List<String>> result = new ArrayList<>();
    for (int i = 0; i < strs.length; i++) {
        List<String> anagrams = new ArrayList<>();
        if (strs[i] != null) {
            char[] sortedWordArray = strs[i].toCharArray();
            Arrays.sort(sortedWordArray);
            anagrams.add(strs[i]);
            for (int j = i + 1; j < strs.length; j++) {
                if (strs[j] != null) {
                    char[] sortedWordArray1 = strs[j].toCharArray();
                    Arrays.sort(sortedWordArray1);
                    if (Arrays.equals(sortedWordArray, sortedWordArray1)) {
                        anagrams.add(strs[j]);
                        strs[j] = null;
                    }
                }
            }
            result.add(anagrams);
        }
    }
    return result;
}

/**
 * Time Complexity: O(n)
 * Space Complexity: O(n)
 */

public static List<List<String>> groupAnagramsOptimised(String[] strs) {
    Map<String, List<String>> anagramMap = new HashMap<>();
    for (String str : strs) {
        char[] chars = str.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        anagramMap.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
    }
    return new ArrayList<>(anagramMap.values());
}
}

```

isPalindrome_7.java

```
package old.LeetCode;

import org.testng.Assert;

public class isPalindrome_7 {
    //A phrase is a palindrome if, after converting all uppercase letters into
    lowercase letters and removing all
    //non-alphanumeric characters, it reads the same forward and backward.
    Alphanumeric characters include letters and numbers.

    public static void main(String[] args) {
        Assert.assertTrue(isPalindrome("A man, a plan, a canal: Panama"));
    }

    /**
     * Time Complexity: O(n)
     * Space Complexity: O(1)
     */

    public static boolean isPalindrome(String s) {
        int left = 0, right = s.length() - 1;

        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }

            if (Character.toLowerCase(s.charAt(left)) !=
                Character.toLowerCase(s.charAt(right))) {
                return false;
            }

            left++;
            right--;
        }
        return true;
    }
}
```

LengthOfLastWord.java

```
package old.LeetCode;

public class LengthOfLastWord {
```

```

public static void main(String[] args) {
    String s="    fly me    to    the moon ";
    System.out.println(lengthOfLastWord(s));
}
public static int lengthOfLastWord(String s) {
    s=s.trim();
    String[] splitStrings=s.split("\\s+");
    int len=splitStrings.length-1;
    return splitStrings[len].length();
}
}

```

MaxAverage_13.java

```

package old.LeetCode;

public class MaxAverage_13 {

    //You are given an integer array nums consisting of n elements, and an integer
    k. Find a contiguous subarray whose length is equal to k that has the maximum
    average value and return this value

    public static void main(String[] args) {
        System.out.println(findMaxAverage(new int[]{1, 12, -5, -6, 50, 3}, 4));
    }

    //    Time: O(n)
    //    Space: O(1)

    public static double findMaxAverage(int[] nums, int k) {
        int n = nums.length;
        double windowSum = 0;
        for (int i = 0; i < k; i++) {
            windowSum += nums[i];
        }
        double maxSum = windowSum;
        for (int i = k; i < n; i++) {
            windowSum += nums[i] - nums[i - k];
            maxSum = Math.max(maxSum, windowSum);
        }
        return maxSum / k;
    }
}

```

MergeSortedArrays.java

```
package old.LeetCode;

public class MergeSortedArrays {
    public static void main(String[] args) {
        int[] nums1_1 = {1, 2, 3, 0, 0, 0};
        int m1 = 3;
        int[] nums2_1 = {2, 5, 6};
        int n1 = 3;
        merge(nums1_1, m1, nums2_1, n1);
        System.out.print("Merged Array 1: ");
        for (int num : nums1_1) {
            System.out.print(num + " ");
        }
    }

    public static void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1; // Pointer for nums1
        int j = n - 1; // Pointer for nums2
        int k = m + n - 1; // Pointer for the merged array (nums1)

        while (i >= 0 && j >= 0) {
            if (nums1[i] > nums2[j]) {
                nums1[k] = nums1[i];
                i--;
            } else {
                nums1[k] = nums2[j];
                j--;
            }
            k--;
        }

        // If there are remaining elements in nums2, copy them to nums1
        while (j >= 0) {
            nums1[k] = nums2[j];
            j--;
            k--;
        }
    }
}
```

MergeSortedLists.java

```
package old.LeetCode;

public class MergeSortedLists {

    public static class ListNode {
        int val;
        ListNode next;
    }
}
```

```
ListNode() {
}

ListNode(int val) {
    this.val = val;
}

ListNode(int val, ListNode next) {
    this.val = val;
    this.next = next;
}

}

public ListNode mergeTwoSortedLists(ListNode list1, ListNode list2) {
    ListNode mergedList = new ListNode();
    ListNode tail = mergedList;

    while (list1 != null && list2 != null) {
        if (list1.val < list2.val) {
            tail.next = list1;
            list1 = list1.next;
        } else {
            tail.next = list2;
            list2 = list2.next;
        }
        tail = tail.next;
    }

    if (list1 != null) {
        tail.next = list1;
    } else {
        tail.next = list2;
    }
    return mergedList.next;
}

public static void main(String[] args) {
    MergeSortedLists solution = new MergeSortedLists();

    // Create sample sorted lists
    ListNode list1 = new ListNode(1, new ListNode(2, new ListNode(5)));
    ListNode list2 = new ListNode(1, new ListNode(3, new ListNode(4)));

    // Merge the lists

    // Print the merged list
    ListNode current = solution.mergeTwoSortedLists(list1, list2);
    while (current != null) {
        System.out.print(current.val + " ");
        current = current.next;
    }
    System.out.println(); // Add a newline for better readability.
}
```



```

//Test case 2: Empty lists
ListNode empty1 = null;
ListNode empty2 = null;

current = solution.mergeTwoSortedLists(empty1, empty2);
while (current != null) {
    System.out.print(current.val + " ");
    current = current.next;
}
System.out.println();

//Test case 3: One empty list
ListNode list3 = new ListNode(5, new ListNode(6, new ListNode(7)));
current = solution.mergeTwoSortedLists(list3, null);
while (current != null) {
    System.out.print(current.val + " ");
    current = current.next;
}
System.out.println();
}
}

```

MissingNumber_6.java

```

package old.LeetCode;

import org.testng.Assert;

public class MissingNumber_6 {
    //Given an array nums containing n distinct numbers in the range [0, n],
    return the only number in the range that is missing from the array.

    public static void main(String[] args) {
        Assert.assertEquals(missingNumber(new int[]{3, 0, 1}),2);
        char a='a';
        if(Character.isAlphabetic(a)){
            System.out.println(true);
        }
    }

    /**
     * Time Complexity: O(n)
     * Space Complexity: O(1)
     */

    public static int missingNumber(int[] nums) {
        int len = nums.length;
        int sum = (len * (len + 1)) / 2;
        int arraySum = 0;
        for (int i : nums) {
            arraySum += i;
        }
    }
}

```

```
    }  
    return sum - arraySum;  
}  
}
```

MoveZeroes_8.java

```
package old.LeetCode;  
  
import java.util.Arrays;  
  
public class MoveZeroes_8 {  
  
    //Given an integer array nums, move all 0's to the end of it while maintaining  
    the relative order of the non-zero elements.  
    //Note that you must do this in-place without making a copy of the array.  
  
    public static void main(String[] args) {  
        moveZeroes(new int[]{0, 1, 0, 3, 12});  
    }  
  
    /**  
     * Time Complexity: O(n)  
     * Space Complexity: O(1)  
     */  
  
    public static void moveZeroes(int[] nums) {  
        int slow = 0;  
  
        for (int fast = 0; fast < nums.length; fast++) {  
            if (nums[fast] != 0) {  
                if (fast != slow) {  
                    int temp = nums[fast];  
                    nums[fast] = nums[slow];  
                    nums[slow] = temp;  
                }  
                slow++;  
            }  
        }  
        System.out.println(Arrays.toString(nums));  
    }  
}
```

RangeSum_5.java

```
package old.LeetCode;
```

```
import org.testng.Assert;
import org.testng.annotations.Test;

public class RangeSum_5 {
    //Given an integer array nums, handle multiple queries of the following type:
    //Calculate the sum of the elements of nums between indices left and right
    inclusive where left <= right.

    /*
    Time Complexity: O(1)
    Space Complexity: O(1)
    */

    static class NumArray {
        private final int[] prefixSums;

        public NumArray(int[] nums) {
            prefixSums = new int[nums.length + 1];
            for (int i = 0; i < nums.length; i++) {
                prefixSums[i + 1] = prefixSums[i] + nums[i];
            }
        }

        public int sumRange(int left, int right) {
            return prefixSums[right + 1] - prefixSums[left];
        }
    }

    @Test
    public void testStandardRange() {
        int[] nums = {-2, 0, 3, -5, 2, -1};
        RangeSum_5.NumArray numArray = new NumArray(nums);

        // Sum of indices 0 to 2: (-2 + 0 + 3) = 1
        Assert.assertEquals(numArray.sumRange(0, 2), 1, "Sum of range [0, 2]
failed");

        // Sum of indices 2 to 5: (3 + -5 + 2 + -1) = -1
        Assert.assertEquals(numArray.sumRange(2, 5), -1, "Sum of range [2, 5]
failed");

        // Sum of indices 0 to 5 (Full Array): -3
        Assert.assertEquals(numArray.sumRange(0, 5), -3, "Sum of full range
failed");
    }

    @Test
    public void testSingleElementRange() {
        int[] nums = {1, 10, 100};
        RangeSum_5.NumArray numArray = new NumArray(nums);

        // If left == right, it should return just that element
        Assert.assertEquals(numArray.sumRange(1, 1), 10, "Single element range [1,
1] failed");
    }
}
```

```

        Assert.assertEquals(numArray.sumRange(0, 0), 1, "Single element range [0,
0] failed");
    }

    @Test
    public void testLargeValues() {
        int[] nums = {1000, -1000, 1000};
        RangeSum_5.NumArray numArray = new NumArray(nums);

        // Sum of [0, 1] should be 0
        Assert.assertEquals(numArray.sumRange(0, 1), 0, "Sum of cancelling values
failed");
    }
}

```

RemoveDuplicates_10.java

```

package old.LeetCode;

import java.util.Arrays;

public class RemoveDuplicates_10 {

    //Given an integer array nums sorted in non-decreasing order, remove the
    duplicates in-place such that each unique element appears only once. The relative
    order of the elements should be kept the same.
    //Consider the number of unique elements in nums to be k. After removing
    duplicates, return the number of unique elements k.
    //The first k elements of nums should contain the unique numbers in sorted order.
    The remaining elements beyond index k - 1 can be ignored.

    public static void main(String[] args) {
        System.out.println(removeDuplicates(new int[]
{0,0,1,1,1,2,2,3,3,4})); //[0,1,2,3,4,_,_,_,_,_] count=5
    }

    public static int removeDuplicates(int[] nums) {
        int count = 0;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[count]) {
                count++;
                nums[count] = nums[i];
            }
        }
        System.out.println("Array after Operations is " + Arrays.toString(nums));
        return count + 1;
    }
}

```

RemoveElement.java

```
package old.LeetCode;

import java.util.Arrays;

public class RemoveElement {
    //Given an integer array nums and an integer val, remove all occurrences of
    val in nums in-place. The order of the elements may be changed. Then return the
    number of elements in nums which are not equal to val.
    public static void main(String[] args) {
        int[] nums = {3, 2, 2, 3};
        System.out.println(removeElement(nums, 3));
    }

    public static int removeElement(int[] nums, int val) {
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[j] = nums[i];
                j++;
            }
        }
        System.out.println(Arrays.toString(Arrays.copyOf(nums, j)));
        return j;
    }
}
```

ReverseString_12.java

```
package old.LeetCode;

import java.util.Arrays;

public class ReverseString_12 {
    //Write a function that reverses a string. The input string is given as an
    array of characters s.

    public static void main(String[] args) {
        char[] arr=new char[]{'h','e','l','l','o'};
        reverseString(arr);
        System.out.println(Arrays.toString(arr));
    }

    public static void reverseString(char[] s) {
        int left=0,right=s.length-1;
        while(left<right){
            char temp=s[left];
            s[left]=s[right];
            s[right]=temp;
        }
    }
}
```

```

        s[right]=temp;
        left++;
        right--;
    }
}
}

```

RomanToInt.java

```

package old.LeetCode;

import java.util.HashMap;
import java.util.Map;

public class RomanToInt {
    public static void main(String[] args) {
        System.out.println(romanToInt("X"));
    }

    public static int romanToInt(String s) {
        Map<Character, Integer> romanMap = new HashMap<>();
        romanMap.put('I', 1);
        romanMap.put('V', 5);
        romanMap.put('X', 10);
        romanMap.put('L', 50);
        romanMap.put('C', 100);
        romanMap.put('D', 500);
        romanMap.put('M', 1000);

        int result = 0;
        int prevValue = 0;

        for (int i = s.length() - 1; i >= 0; i--) {
            char currentChar = s.charAt(i);
            int currentValue = romanMap.get(currentChar);

            if (currentValue < prevValue) {
                result -= currentValue;
            } else {
                result += currentValue;
            }
            prevValue = currentValue;
        }
        return result;
    }
}

```

SearchInsertPosition.java

```
package old.LeetCode;

import org.testng.annotations.Test;

import static org.testng.Assert.assertEquals;

public class SearchInsertPosition {

    //Given a sorted array of distinct integers and a target value, return the
    index if the target is found. If not, return the index where it would be if it
    were inserted in order.

    public int searchInsert(int[] nums, int target) {
        int len = nums.length;
        for (int i = 0; i < len; i++) {
            if (nums[i] == target) {
                return i;
            } else if (nums[i] > target) {
                return i;
            }
        }
        return len;
    }

    public int searchInsertBinary(int[] nums, int target) {
        int left = 0;
        int right = nums.length;
        while (left < right) {
            int mid = (left + right) / 2;
            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid;
            }
        }
        return left;
    }

    @Test
    public void testExample1() {
        int[] nums = {1, 3, 5, 6};
        int target = 5;
        assertEquals(searchInsert(nums, target), 2);
        assertEquals(searchInsertBinary(nums, target), 2);
    }

    @Test
    public void testExample2() {
        int[] nums = {1, 3, 5, 6};
        int target = 2;
```

```
        assertEquals(searchInsert(nums, target), 1);
        assertEquals(searchInsertBinary(nums, target), 1);
    }

    @Test
    public void testExample3() {
        int[] nums = {1, 3, 5, 6};
        int target = 7;
        assertEquals(searchInsert(nums, target), 4);
        assertEquals(searchInsertBinary(nums, target), 4);
    }

    @Test
    public void testExample4() {
        int[] nums = {1, 3, 5, 6};
        int target = 0;
        assertEquals(searchInsert(nums, target), 0);
        assertEquals(searchInsertBinary(nums, target), 0);
    }

    @Test
    public void testExample5() {
        int[] nums = {1};
        int target = 0;
        assertEquals(searchInsert(nums, target), 0);
        assertEquals(searchInsertBinary(nums, target), 0);
    }

    @Test
    public void testExample6() {
        int[] nums = {1};
        int target = 2;
        assertEquals(searchInsert(nums, target), 1);
        assertEquals(searchInsertBinary(nums, target), 1);
    }

    @Test
    public void testExample7() {
        int[] nums = {1, 2, 3, 4, 5};
        int target = 3;
        assertEquals(searchInsert(nums, target), 2);
        assertEquals(searchInsertBinary(nums, target), 2);
    }

    @Test
    public void testExample8() {
        int[] nums = {1, 2, 3, 4, 5};
        int target = 6;
        assertEquals(searchInsert(nums, target), 5);
        assertEquals(searchInsertBinary(nums, target), 5);
    }

    @Test
    public void testExample9() {
```



```
        int[] nums = {1, 2, 3, 4, 5};
        int target = 0;
        assertEquals(searchInsert(nums, target), 0);
        assertEquals(searchInsertBinary(nums, target), 0);
    }
}
```

SortedSquares_11.java

```
package old.LeetCode;

import java.util.Arrays;

public class SortedSquares_11 {

    //Given an integer array nums sorted in non-decreasing order, return an array
    of the squares of each number sorted in non-decreasing order.
    //Time Complexity:O(n)
    //Space Complexity:O(n)

    public static void main(String[] args) {
        System.out.println(Arrays.toString(sortedSquares(new int[]{-4, -1, 0, 3,
10})));
    }

    public static int[] sortedSquares(int[] nums) {
        int length = nums.length;
        int[] squares = new int[length];

        int left = 0;
        int right = length - 1;
        int p = length - 1;

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                squares[p] = leftSquare;
                left++;
            } else {
                squares[p] = rightSquare;
                right--;
            }
            p--;
        }

        return squares;
    }
}
```

```
}
```

SquareRoot.java

```
package old.LeetCode;

import org.testng.Assert;

public class SquareRoot {
    public static void main(String[] args) {
        Assert.assertEquals(mySqrt(9), 3);
    }

    public static int mySqrt(int x) {
        if (x == 0) {
            return 0;
        }

        long left = 1;
        long right = x;
        long ans = 1;

        while (left <= right) {
            long mid = (left + right) / 2;
            if (mid * mid <= x) {
                ans = mid;
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return (int) ans;
    }
}
```

TwoSum_2.java

```
package old.LeetCode;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

public class TwoSum_2 {

    /**
     * Time Complexity: O(n)
```

```

    * Space Complexity: O(n)
    */

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int target = 9;
        int[] ans=twoSum(nums,target);
        System.out.println(Arrays.toString(ans));
    }

    public static int[] twoSum(int[] nums, int target) {
        Map<Integer,Integer> hs=new HashMap<>();
        int[] ans=new int[2];
        for(int i=0;i<nums.length;i++){
            int counter=target-nums[i];
            if(hs.containsKey(counter)){
                ans[0]=i;
                ans[1]=hs.get(counter);
                break;
            }else{
                hs.put(nums[i],i);
            }
        }
        return ans;
    }
}

```

TwoSum_9.java

```

package old.LeetCode;

import java.util.Arrays;

public class TwoSum_9 {
    //Given a 1-indexed array of integers numbers that is already sorted in non-
    decreasing order, find two numbers such that they add up to a specific target
    number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <=
    index1 < index2 <= numbers.length. Return the indices of the two numbers index1
    and index2, each incremented by one, as an integer array [index1, index2] of
    length 2.

    public static void main(String[] args) {
        int[] nums = {2, 7, 11, 15};
        int target = 9;
        int[] ans = twoSum(nums, target);
        System.out.println(Arrays.toString(ans));
    }

    /**
     * Time Complexity: O(n)

```

```
* Space Complexity: O(1)
*/

public static int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;

    while (left < right) {
        int sum = numbers[left] + numbers[right];

        if (sum == target) {
            return new int[]{left + 1, right + 1};
        } else if (sum > target) {
            right--;
        } else {
            left++;
        }
    }
    return new int[]{-1, -1};
}
```

ValidAnagram_3.java

```
package old.LeetCode;

import org.testng.Assert;

import java.util.Arrays;

public class ValidAnagram_3 {

    public static void main(String[] args) {
        Assert.assertTrue(isAnagram("anagram", "nagaram"));
        Assert.assertTrue(isAnagramOptimised("anagram", "nagaram"));
    }

    /**
     * Time Complexity: O(nlogn)
     * Space Complexity: O(n)
     */

    public static boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;

        char[] arrayS = s.toCharArray();
        char[] arrayT = t.toCharArray();

        Arrays.sort(arrayS);
        Arrays.sort(arrayT);
    }
}
```

```
        return Arrays.equals(arrayS, arrayT);
    }

    /**
     * Time Complexity: O(n)
     * Space Complexity: O(1)
     */

    public static boolean isAnagramOptimised(String s, String t) {
        if (s.length() != t.length()) return false;

        int[] counts = new int[26];

        for (int i = 0; i < s.length(); i++) {
            counts[s.charAt(i) - 'a']++;
            counts[t.charAt(i) - 'a']--;
        }

        for (int count : counts) {
            if (count != 0) return false;
        }

        return true;
    }
}
```

Topic: LinkedIn

Code Solutions

AddOneToNumberTest.java

```
package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

// QUESTION: Given a non-negative integer represented as an array of digits, add
// one to the integer.
// The digits are stored such that the most significant digit is at the head of
// the list, and each
// element in the array contains a single digit. You may assume the integer does
// not contain any
```

```
// leading zero, except the number 0 itself.
// Example: Input: [1,2,3] -> Output: [1,2,4]
//           Input: [9,9,9] -> Output: [1,0,0,0]

public class AddOneToNumberTest {

    public static int[] addOne(int[] arr) {
        List<Integer> result = new ArrayList<>();
        int carry = 1;
        for (int i = arr.length - 1; i >= 0; i--) {
            int plusOne = arr[i] + carry;
            if (plusOne > 9) {
                result.add(0);
                carry = plusOne / 10;
            } else {
                result.add(plusOne);
                carry = 0;
            }
        }
        if (carry > 0) {
            result.add(carry);
        }
        Collections.reverse(result);
        return result.stream().mapToInt(i -> i).toArray();
    }

    @Test
    public void test() {
        Assert.assertEquals(addOne(new int[]{1, 2, 4}), new int[]{1, 2, 5});
        Assert.assertEquals(addOne(new int[]{1, 2, 9}), new int[]{1, 3, 0});
        Assert.assertEquals(addOne(new int[]{9, 9, 9}), new int[]{1, 0, 0, 0});
        Assert.assertEquals(addOne(new int[]{9}), new int[]{1, 0});
        Assert.assertEquals(addOne(new int[]{5}), new int[]{6});
    }
}
```

AnagramCheck.java

```
package old.Linkedin;

import java.util.Arrays;

// QUESTION: Given two strings, write a function to determine if they are anagrams
// of each other.
// An anagram is a word or phrase formed by rearranging the letters of a different
// word or phrase,
// typically using all the original letters exactly once. The comparison should be
// case-insensitive
// and ignore spaces.
// Example: Input: "Saurav is Singh Hexaware", "Singh Saurav Hexaware is" ->
```

```
Output: true
public class AnagramCheck {
    public static void main(String[] args) {
        String str1 = "Saurav is Singh Hexaware";
        String str2 = "Singh Saurav Hexaware is";

        if (areAnagrams(str1, str2)) {
            System.out.println("Anagram");
        } else {
            System.out.println("No");
        }
    }

    static boolean areAnagrams(String str1, String str2) {
        if (str1.length() != str2.length())
            return false;

        str1 = str1.replaceAll("\\s", "").toLowerCase();
        str2 = str2.replaceAll("\\s", "").toLowerCase();

        char[] arr1 = str1.toCharArray();
        char[] arr2 = str2.toCharArray();

        Arrays.sort(arr1);
        Arrays.sort(arr2);

        return Arrays.equals(arr1, arr2);
    }
}
```

ArmstrongNumbers.java

```
package old.Linkedin;

// QUESTION: An Armstrong number (also known as narcissistic number) is a number
// that is equal to
// the sum of its own digits each raised to the power of the number of digits.
// Write a function to check if a given number is an Armstrong number.
// Example: Input: 153 -> Output: true ( $1^3 + 5^3 + 3^3 = 153$ )
//          Input: 370 -> Output: true ( $3^3 + 7^3 + 0^3 = 370$ )
public class ArmstrongNumbers {

    public static void main(String[] args) {
        int num1 = 153;
        if (isArmstrong(num1)) {
            System.out.println("yes");
        } else {
            System.out.println("No");
        }
    }
}
```

```

    int first = 50;
    int second = 50000000;
    for (int i = first; i <= second; i++) {
        if (isArmstrong(i)) {
            System.out.println(i);
        }
    }
}

static boolean isArmstrong(int num) {
    int originalNum = num;
    int digits = countDigits(num);
    int pow = 0;

    while (num != 0) {
        int rem = num % 10;
        pow += (int) Math.pow(rem, digits);
        if (pow > originalNum) {
            return false; // Early termination
        }
        num /= 10;
    }

    return pow == originalNum;
}

static int countDigits(int num) {
    int digits = 0;
    while (num != 0) {
        num /= 10;
        digits++;
    }
    return digits;
}
}

```

ArrayElementFrequency.java

```

package old.Linkedin;

import java.util.HashMap;

// QUESTION: Given an array of integers, count the frequency of each element and
// print the result.
// Example: Input: [1, 2, 3, 4, 2, 1, 2, 3, 4, 2, 3] -> Output: 1: 2, 2: 4, 3: 3,
// 4: 2

public class ArrayElementFrequency {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 2, 1, 2, 3, 4, 2, 3};
        HashMap<Integer, Integer> frequencyMap = new HashMap<>();
    }
}

```



```

        for (int element : arr) {
            frequencyMap.put(element, frequencyMap.getOrDefault(element, 0) + 1);
        }
        for (int element : frequencyMap.keySet()) {
            System.out.println(element + ": " + frequencyMap.get(element));
        }
    }
}

```

ArrayListTest.java

```

package old.Linkedin;
import java.util.Arrays;

// QUESTION: Given an array of strings, remove duplicate characters from each
// string while
// preserving the order of first occurrence.
// Example: Input: ["hello", "helloishi", "mastercard"] -> Output: ["helo",
// "helois", "mstercd"]
public class ArrayListTest {
    public static void main(String[] args) {
        String[] words = {"hello", "helloishi", "mastercard"};
        String[] result = new String[words.length];

        for (int i = 0; i < words.length; i++) {
            result[i] = removeDuplicates(words[i]);
        }

        System.out.println(Arrays.toString(result));
    }

    private static String removeDuplicates(String word) {
        StringBuilder sb = new StringBuilder();
        for (char c : word.toCharArray()) {
            if (sb.indexOf(String.valueOf(c)) == -1) {
                sb.append(c);
            }
        }
        return sb.toString();
    }
}

```

BinarySearch.java

```

package old.Linkedin;

import java.util.Arrays;

```

```
// QUESTION: Implement binary search to find a target element in a sorted array.
// Return the index if found, otherwise return -1.
// Example: Input: [2, 5, 6, 7, 9], target: 7 -> Output: 3
//           Input: [2, 5, 6, 7, 9], target: 8 -> Output: -1
public class BinarySearch {
    public static void main(String[] args) {
        int[] arr = {5, 6, 7, 9, 2};
        Arrays.sort(arr);

        int num = 7;
        int result = binarySearch(arr, num);

        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            System.out.println("Element not found in the array.");
        }
    }

    static int binarySearch(int[] arr, int num) {
        int len = arr.length;
        int max = len - 1;
        int min = 0;

        while (max >= min) {
            int mid = (max + min) / 2;

            if (arr[mid] == num) {
                return mid;
            } else if (arr[mid] > num) {
                max = mid - 1;
            } else {
                min = mid + 1;
            }
        }

        return -1; // Element not found
    }
}
```

BubbleSort.java

```
package old.Linkedin;

// QUESTION: Implement bubble sort algorithm to sort an array of integers in
// ascending order.
// Example: Input: [9, 8, 7, 4, 5, 6, 2, 1, 3] -> Output: [1, 2, 3, 4, 5, 6, 7, 8,
// 9]
public class BubbleSort {
```

```

public static void main(String[] args) {
    int[] array = {9, 8, 7, 4, 5, 6, 2, 1, 3};
    int tempValue;
    for (int i = 0; i < array.length; i++) {
        for (int j = i + 1; j < array.length; j++) {
            if (array[i] > array[j]) {
                tempValue = array[i];
                array[i] = array[j];
                array[j] = tempValue;
            }
        }
    }
    System.out.print("Sorted Array: ");
    for (int i : array) {
        System.out.print(i + " ");
    }
}

```

CharacterFrequencyTest.java

```

package old.Linkedin;

public class CharacterFrequencyTest {
    // QUESTION: Given a string, calculate the frequency of each character and
    // print the results.
    // Example: Input: "Picture Perfect" -> Output: P: 2, i: 1, c: 2, t: 2, u: 1,
    // r: 2, e: 3, f: 1, : 1
    //t:O(n),s:O(1)
    public static void main(String[] args) {
        calculateFrequency("Picture Perfect");
    }

    public static void calculateFrequency(String str) {
        int[] countArr = new int[256];
        char[] arr = str.toCharArray();
        for (char c : arr) {
            countArr[c]++;
        }
        for (int i = 0; i < countArr.length; i++) {
            if (countArr[i] > 0) System.out.println((char) i + " :" +
countArr[i]);
        }
    }
}

```

CharsRepeatedTwice.java

```

package old.Linkedin;

// QUESTION: Given a string, find all characters that appear at least twice and
// print their frequency.
// Example: Input: "aabccdad" -> Output: a: 3, c: 2, d: 2
public class CharsRepeatedTwice {
    public static void main(String[] args) {
        String str = "aabccdad";//a,c,d
        int[] counter=new int[26];
        for(char c:str.toCharArray()){
            counter[c-'a']++;
        }
        for(int i=0;i<counter.length;i++){
            if(counter[i]>=2){
                System.out.println((char)(i+'a')+" : "+counter[i]);
            }
        }
    }
}

```

CommonStringTest.java

```

package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.util.*;

// QUESTION: Given two arrays of strings, find the common elements between them.
// Example: Input: ["apple", "banana", "single", "Laxmi", "Raghu"], ["june",
// "july", "apple", "april", "Laxmi"]
// Output: ["apple", "Laxmi"]
public class CommonStringTest {
    //Time Complexity: O(n + m)
    //Space Complexity: O(n + m)

    @Test
    public void test(){
        String[] a1 = {"apple", "banana", "single", "Laxmi", "Raghu"};
        String[] a2 = {"june", "july", "apple", "april", "Laxmi"};
        Assert.assertEquals(commonString(a1,a2),List.of("apple", "Laxmi"));
    }

    public List<String> commonString(String[] a1,String[] a2){
        Set<String> a2Set = new HashSet<>(Arrays.asList(a2));
        List<String> result = new ArrayList<>();
        for (String ele : a1) {
            if (a2Set.contains(ele)) {
                result.add(ele);
            }
        }
    }
}

```

```

    }
    }
    return result;
}
}

```

ComplexStringReverseTest.java

```

package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

// QUESTION: Given a string, reverse it while maintaining the original positions
// of spaces.
// Example: Input: "this is a phone" -> Output: "enoh pa s isiht"
public class ComplexStringReverseTest {

    @Test
    public void test(){
        Assert.assertEquals(reverseString("this is a phone"),"enoh pa s isiht");
    }

    public static String reverseString(String str) {
        char[] arr = str.toCharArray();
        int left = 0;
        int right = arr.length - 1;

        while (left < right) {
            if (arr[left] == ' ') {
                left++;
            } else if (arr[right] == ' ') {
                right--;
            } else {
                char temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;
                left++;
                right--;
            }
        }
        return new String(arr);
    }
}

```

ContainerwithMaxWaterTest.java

```

package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

// QUESTION: You are given an integer array height of length n. There are n
// vertical lines drawn such
// that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two
// lines that together
// with the x-axis form a container, such that the container contains the most
// water. Return the
// maximum amount of water a container can store.
// Example: Input: [1,8,6,2,5,4,8,3,7] -> Output: 49
public class ContainerwithMaxWaterTest {
    @Test
    public void testMaxArea() {
        List<Integer> height = new ArrayList<>(Arrays.asList(1, 8, 6, 2, 5, 4, 8,
3, 7));
        Assert.assertEquals(calculateMaxArea(height), 49);
    }

    public int calculateMaxArea(List<Integer> height) {
        int i = 0, j = height.size() - 1;
        int maxArea = Integer.MIN_VALUE;
        while (i <= j) {
            int width = j - i;
            int length = Math.min(height.get(i), height.get(j));
            maxArea = Math.max(maxArea, width * length);

            if (height.get(i) < height.get(j)) {
                i++;
            } else {
                j--;
            }
        }
        return maxArea;
    }
}

```

CustomLinkedList.java

```

package old.Linkedin;

// QUESTION: Implement a custom linked list with the following operations:
// - Add a node at the end
// - Remove the first node

```

```
// - Remove the last node
// - Remove a node at a specific index
// - Delete all nodes with a specific key value
// - Print the list
public class CustomLinkedList {
    int length;
    private Node head;
    private Node tail;

    public CustomLinkedList(int value) {
        Node node = new Node(value);
        head = node;
        tail = node;
        length = 1;
    }

    public void printList() {
        Node temp = head;
        while (temp.next != null) {
            System.out.println(temp.value);
            temp = temp.next;
        }
        System.out.println(temp.value);
    }

    public void add(int value) {
        Node node = new Node(value);
        if (length < 1) {
            head = node;
        } else {
            tail.next = node;
        }
        tail = node;
        length++;
    }

    public void removeFirst() {
        if (length <= 0)
            return;
        Node temp = head;
        head = temp.next;
        temp.next = null;
        length--;
    }

    public void removeLast() {
        if (length <= 0)
            return;
        Node temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        tail = temp;
        tail.next = null;
    }
}
```

```
        length--;  
    }  
  
    public void remove(int index) {  
        if (length <= 0)  
            return;  
        Node temp = head;  
        for (int i = 0; i < index; i++) {  
            temp = temp.next;  
        }  
        Node toDelete = temp.next;  
        temp.next = toDelete.next;  
        toDelete.next = null;  
        length--;  
    }  
  
    public void deleteKey(int key) {  
        Node prev = null;  
        Node curr = head;  
  
        while (curr != null) {  
            if (curr.value == key) {  
                if (prev == null) {  
                    head = curr.next;  
                } else {  
                    prev.next = curr.next;  
                }  
                curr = curr.next;  
            } else {  
                prev = curr;  
                curr = curr.next;  
            }  
        }  
    }  
  
    static class Node {  
        int value;  
        Node next;  
  
        public Node(int value) {  
            this.value = value;  
            next = null;  
        }  
    }  
}
```

Factorial.java


```
package old.Linkedin;

public class Factorial {
    // QUESTION: Write a recursive function to calculate the factorial of a given
    // number.
    // Example: Input: 5 -> Output: 120 (5 * 4 * 3 * 2 * 1)
    //S/T:n
    public static void main(String[] args) {
        int number = 5;
        int result = factorial(number);
        System.out.println("Factorial of " + number + " is " + result);
    }

    public static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }
}
```

FibonacciSeries.java

```
package old.Linkedin;

public class FibonacciSeries {
    // QUESTION: Generate the first n numbers of the Fibonacci series.
    // The Fibonacci sequence is a series where each number is the sum of the two
    // preceding ones.
    // Example: Input: 10 -> Output: 0 1 1 2 3 5 8 13 21 34

    public static void main(String[] args) {
        int firstNum = 0, secondNum = 1, nextNum, count = 10;
        System.out.print(firstNum + " " + secondNum);
        for (int i = 2; i < count; i++) {
            nextNum = firstNum + secondNum;
            System.out.print(" " + nextNum);
            firstNum = secondNum;
            secondNum = nextNum;
        }
    }
}
```

GroupAnagrams.java

```

package old.Linkedin;

import java.util.*;

// QUESTION: Given an array of strings, group the anagrams together. An anagram is
// a word or phrase
// formed by rearranging the letters of a different word or phrase.
// Example: Input: ["eat", "tea", "tan", "ate", "nat", "bat"]
//           Output: [["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]
public class GroupAnagrams {
    public static void main(String[] args) {
        String[] arr = {"eat", "tea", "tan", "ate", "nat", "bat"};
        List<List<String>> result = groupAnagrams(arr);
        for (List<String> group : result) {
            System.out.println(group);
        }
    }

    public static List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> anagramMap = new HashMap<>();
        for (String str : strs) {
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            anagramMap.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
        }
        return new ArrayList<>(anagramMap.values());
    }
}

```

IntegerFrequency.java

```

package old.Linkedin;

public class IntegerFrequency {
    // QUESTION: Given an array of integers, count the frequency of each element
    // and print the results.
    // Do not use additional data structures like HashMap.
    // Example: Input: [1, 2, 3, 1, 2, 3, 4, 44] -> Output: Count of 1 is 2, Count
    // of 2 is 2, etc.
    //Time:O(n^2)
    //Space:O(n)
    public static void countOccurrences(int[] arr) {
        boolean[] visited = new boolean[arr.length];
        for (int i = 0; i < arr.length; i++) {
            if (!visited[i]) {
                int count = 1; // Include the current element itself
                for (int j = i + 1; j < arr.length; j++) {
                    if (arr[i] == arr[j]) {
                        count++;
                    }
                }
            }
        }
    }
}

```

```

                visited[j] = true;
            }
        }
        System.out.println("Count of " + arr[i] + " is " + count);
    }
}

public static void main(String[] args) {
    int[] arr = new int[]{1, 2, 3, 1, 2, 3, 4, 44};
    countOccurrences(arr);
}
}

```

IntegerReverse.java

```

package old.Linkedin;

// QUESTION: Given an integer, reverse its digits and return the result.
// Example: Input: 12345 -> Output: 54321
//           Input: 123 -> Output: 321
public class IntegerReverse {
    public static void main(String[] args) {
        int num = 12345;
        System.out.println(reverseInteger(num));

        int num1 = 123;
        System.out.println(reverseInteger(num1));
    }

    static int reverseInteger(int num) {
        int rev = 0;
        while (num != 0) {
            rev = rev * 10 + num % 10;
            num = num / 10;
        }
        return rev;
    }
}

```

IsomorphicStrings.java

```

package old.Linkedin;

import org.testng.Assert;
import java.util.HashMap;
import java.util.HashSet;

```

```
// QUESTION: Given two strings s and t, determine if they are isomorphic. Two
// strings are isomorphic
// if the characters in s can be replaced to get t. All occurrences of a character
// must be replaced
// with another character while preserving the order of characters. No two
// characters may map to
// the same character, but a character may map to itself.
// Example: Input: "aab", "xxy" -> Output: true
//           Input: "bar", "foo" -> Output: false
public class IsomorphicStrings {
    public static boolean areIsomorphic(String str1, String str2) {
        if (str1.length() != str2.length()) {
            return false;
        }
        HashMap<Character, Character> map = new HashMap<>();
        HashSet<Character> mappedValues = new HashSet<>();
        for (int i = 0; i < str1.length(); i++) {
            char c1 = str1.charAt(i);
            char c2 = str2.charAt(i);
            if (map.containsKey(c1) && map.get(c1) != c2) {
                return false;
            }
            if (!map.containsKey(c1)) {
                if (mappedValues.contains(c2)) {
                    return false;
                }
                map.put(c1, c2);
                mappedValues.add(c2);
            }
        }
        return true;
    }

    public static void main(String[] args) {
        Assert.assertTrue(areIsomorphic("aab", "xxy"));
        Assert.assertFalse(areIsomorphic("bar", "foo"));
        Assert.assertTrue(areIsomorphic("paper", "title"));
        Assert.assertTrue(areIsomorphic("abc", "xyz"));
    }
}
```

KthLargest.java

```
package old.Linkedin;

import java.util.PriorityQueue;

public class KthLargest {
    // QUESTION: Given an array of integers and a number k, find the kth largest
    // element in the array.
    // Example: Input: [3, 2, 1, 5, 6, 4, 7], k = 3 -> Output: 5
}
```

```
//Time Complexity: O(nlogn)
//Space Complexity: O(n)
public static void main(String[] args) {
    int[] arr = {3, 2, 1, 5, 6, 4, 7};
    System.out.println("Third largest number is: " + kthLargest(arr,3));
}

public static int kthLargest(int[] arr, int k) {
    PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();
    for (int val : arr) {
        priorityQueue.add(val);
        if (priorityQueue.size() > k)
            priorityQueue.poll();
    }
    return priorityQueue.peek();
}
}
```

LargestElement.java

```
package old.Linkedin;

public class LargestElement {
    // QUESTION: Given an array of integers, find the largest element.
    // Example: Input: [1, 2, 3, 1, 2, 3, 4, 5] -> Output: 5
    //Space/Time:O(n)
    public static void main(String[] args) {
        int[] arr = new int[]{1, 2, 3, 1, 2, 3, 4, 5};
        int max = Integer.MIN_VALUE;
        for (int i : arr) {
            if (i > max)
                max = i;
        }
        System.out.println(max);
    }
}
```

LargestUniqueNumberTest.java

```
package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

// QUESTION: Given an array of integers, return the largest integer that occurs
// only once. If no
// such integer exists, return -1.
```

```
// Example: Input: [1, 2, 2, 3, 3, 4, 4, 4] -> Output: 1
//           Input: [2, 2, 2, 2] -> Output: -1
public class LargestUniqueNumberTest {

    public static Integer findLargestUniqueNumber(int[] nums) {
        final int kMax = 1000;
        int[] count = new int[kMax + 1];

        for (int num : nums)
            count[num]++;

        for (int num = kMax; num >= 0; num--)
            if (count[num] == 1)
                return num;

        return -1;
    }

    @Test
    public void largestUniqueTest() {
        Assert.assertEquals(findLargestUniqueNumber(new int[]{2, 2, 2, 2}), -1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 2, 2, 3, 3, 4, 4, 4}), 1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 2, 3, 4, 5}), Integer.valueOf(5));
        Assert.assertEquals(findLargestUniqueNumber(new int[]{1, 1, 2, 2, 3, 4}), Integer.valueOf(4));
        Assert.assertEquals(findLargestUniqueNumber(new int[]{4, 3, 3, 2, 2, 1}), Integer.valueOf(4));
        Assert.assertEquals(findLargestUniqueNumber(new int[]{3, 1, 4, 1, 5, 5}), Integer.valueOf(4));
        Assert.assertEquals(findLargestUniqueNumber(new int[]{}), -1);
        Assert.assertEquals(findLargestUniqueNumber(new int[]{99}), Integer.valueOf(99));
    }
}
```

LeapYear.java

```
package old.Linkedin;

public class LeapYear {
    // QUESTION: Write a function to determine if a given year is a leap year.
    // A leap year is divisible by 4, except for century years which must be
    // divisible by 400.
    // Example: Input: 2000 -> Output: true
    //           Input: 1900 -> Output: false

    public static void main(String[] args) {
        System.out.println(isLeapYear(1900));
    }
}
```

```

    }

    public static boolean isLeapYear(int year) {
        return ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0)));
    }
}

```

LinkedListDeleteAllChar.java

```

package old.Linkedin;

public class LinkedListDeleteAllChar {
    // QUESTION: Given a linked list and a key, delete all occurrences of the
    // given key in the linked list.
    // Example: Input: 3 -> 3 -> 6 -> 9 -> 3 -> NULL, key = 3
    //           Output: 6 -> 9 -> NULL

    public static void main(String[] args) {
        CustomLinkedList list = new CustomLinkedList(10);
        list.add(20);
        list.add(30);
        list.add(10);
        list.printList();
        System.out.println("-----");
        list.deleteKey(10);
        list.printList();
    }
}

```

LongestCommonPrefix.java

```

package old.Linkedin;

// QUESTION: Given an array of strings, find the longest common prefix string
// amongst the array.
// If there is no common prefix, return an empty string.
// Example: Input: ["flower", "flow", "flight"] -> Output: "fl"
public class LongestCommonPrefix {
    public static void main(String[] args) {
        String[] strs = {"flower", "flow", "flight"};
        System.out.println(longestCommonPrefix(strs));
    }

    static String longestCommonPrefix(String[] arr) {
        String result = arr[0];
        for (int i = 1; i < arr.length; i++) {
            int len = commonPrefixLen(result, arr[i]);
            result = result.substring(0, len);
        }
    }
}

```

```

        return result;
    }
    static int commonPrefixLen(String a, String b) {
        int commonLength=0;
        int minL=Math.min(a.length(),b.length());
        for(int i=0;i<minL;i++){
            if(a.charAt(i)==b.charAt(i)){
                commonLength++;
            }else{
                break;
            }
        }
        return commonLength;
    }
}

```

LongestPalindromicSubstring.java

```

package old.Linkedin;

public class LongestPalindromicSubstring {
    // QUESTION: Given a string, find the longest palindromic substring within it.
    // A palindrome is a word that reads the same backward as forward.
    // Example: Input: "babad" -> Output: "bab" or "aba"
    //           Input: "cbbd" -> Output: "bb"

    public static void main(String[] args) {
        String str = "babad";
        System.out.println("Longest Palindromic Substring: " +
            findLongestPalindromeSubstring(str));
        str = "cbbd";
        System.out.println("Longest Palindromic Substring: " +
            findLongestPalindromeSubstring(str));
    }

    static boolean isPalindrome(String str, int start, int end) {
        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            } else {
                start++;
                end--;
            }
        }
        return true;
    }

    static String findLongestPalindromeSubstring(String s) {
        int maxLength = 1;
        String longestPalindrome = String.valueOf(s.charAt(0));
    }
}

```



```

        for (int i = 0; i < s.length(); i++) {
            for (int j = 0; j < s.length(); j++) {
                if ((j - i) > maxLength && isPalindrome(s, i, j)) {
                    maxLength = j - i;
                    longestPalindrome = s.substring(i, j + 1);
                }
            }
        }
        return longestPalindrome;
    }
}

```

LongestUniqueCharSubstring.java

```

package old.Linkedin;

import java.util.HashSet;
import java.util.Set;

public class LongestUniqueCharSubstring {
    // QUESTION: Given a string, find the length of the longest substring without
    // repeating characters.
    // Example: Input: "GEEKSFORGEEEKS" -> Output: "EKSFORG" (length 7)
    //Time Complexity: O(n)
    //Space Complexity: O(min(n, m))
    static String longestSubStrWithoutRepetition(String str) {
        Set<Character> hs = new HashSet<>();
        int j = 0;
        int maxLength = 0;
        int startIndex = 0;

        for (int i = 0; i < str.length(); i++) {
            while (hs.contains(str.charAt(i))) {
                hs.remove(str.charAt(j));
                j++;
            }
            hs.add(str.charAt(i));

            if (i - j + 1 > maxLength) {
                maxLength = i - j + 1;
                startIndex = j;
            }
        }
        return str.substring(startIndex, startIndex + maxLength);
    }

    public static void main(String[] args) {
        String s = "GEEKSFORGEEEKS";
        System.out.println(longestSubStrWithoutRepetition(s));
    }
}

```

```

    }
}

```

MaxAverageSubarray.java

```

package old.Linkedin;

// QUESTION: Given an array of integers and a number k, find the contiguous
// subarray of length k
// that has the maximum average value and return that average.
// Example: Input: [1, 12, -5, -6, 50, 3], k = 4 -> Output: 12.75
public class MaxAverageSubarray {
    public static void main(String[] args) {
        int[] arr = {1, 12, -5, -6, 50, 3};
        System.out.println(getMaxAverage(arr, 4)); // Output: 12.75
        System.out.println(getMaxAverage(arr, 3)); // Output: 15.666...
    }

    private static double getMaxAverage(int[] arr, int k) {
        if (arr == null || arr.length < k) {
            throw new IllegalArgumentException("Invalid input");
        }
        double currentSum = 0;
        for (int i = 0; i < k; i++) {
            currentSum += arr[i];
        }
        double maxWindowSum = currentSum;
        for (int i = k; i < arr.length; i++) {
            currentSum = currentSum + arr[i] - arr[i - k];
            maxWindowSum = Math.max(maxWindowSum, currentSum);
        }
        return maxWindowSum / k;
    }
}

```

MaxMinOccurence.java

```

package old.Linkedin;

public class MaxMinOccurence {
    // QUESTION: Given a string, find the character with the maximum frequency and the
    // character
    // with the minimum frequency (excluding characters that don't appear).
    // Example: Input: "TeeeesstAutoooooooooomationn" -> Output: o: 8, m: 1

    public static void main(String[] args) {
        maxMinOccurence();
    }
}

```

```

static void maxMinOccurence() {
    int[] counter = new int[256];
    for (char c : "TeeeesstAutoooooomationn".toCharArray()) {
        counter[c]++;
    }

    int maxCount = Integer.MIN_VALUE;
    int minCount = Integer.MAX_VALUE;
    char maxChar = ' ';
    char minChar = ' ';

    for (int i = 0; i < counter.length; i++) {
        if (counter[i] > maxCount) {
            maxCount = counter[i];
            maxChar = (char) i;
        }
        if (counter[i] >= 1 && counter[i] < minCount) {
            minCount = counter[i];
            minChar = (char) i;
        }
    }
    System.out.println(maxChar + ": " + maxCount);
    System.out.println(minChar + ": " + minCount);
}
}

```

MaxSumSubArray.java

```

package old.Linkedin;

public class MaxSumSubArray {
    // QUESTION: Given an array of integers, find the maximum sum of any
    // contiguous subarray.
    // Example: Input: [1, 12, -5, -6, 50, 3] -> Output: 55 (12 + (-5) + (-6) + 50
    // + 3)
    public static void main(String[] args) {
        int[] arr = {1, 12, -5, -6, 50, 3};
        int maxSum = Integer.MIN_VALUE;
        int currentSum = 0;
        for (int i : arr) {
            currentSum += i;
            maxSum = Math.max(maxSum, currentSum);
            if (currentSum < 0) {
                currentSum = 0;
            }
        }
        System.out.println("Maximum Sum of Contiguous Subarray: " + maxSum);
    }
}

```

MergeSortedArrays.java

```
package old.Linkedin;

import java.util.Arrays;

public class MergeSortedArrays {
    // QUESTION: You are given two integer arrays nums1 and nums2, sorted in non-
    // decreasing order, and two
    // integers m and n, representing the number of elements in nums1 and nums2
    // respectively. Merge nums1
    // and nums2 into a single array sorted in non-decreasing order. The final
    // sorted array should be stored
    // inside the array nums1. nums1 has a length of m + n, where the first m
    // elements denote the elements
    // that should be merged, and the last n elements are set to 0 and should be
    // ignored.
    // Example: Input: nums1 = [1,2,3,4,0,0,0], m = 4, nums2 = [2,5,6], n = 3
    //           Output: nums1 = [1,2,2,3,4,5,6]
    public static void main(String[] args) {
        int[] nums1 = {1, 2, 3, 4, 0, 0, 0}; // nums1 has extra space for nums2
        int[] nums2 = {2, 5, 6};
        merge(nums1, 4, nums2, 3);
        System.out.println(Arrays.toString(nums1));
    }

    static void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = m + n - 1;

        while (i >= 0 && j >= 0) {
            if (nums1[i] > nums2[j]) {
                nums1[k] = nums1[i];
                i--;
            } else {
                nums1[k] = nums2[j];
                j--;
            }
            k--;
        }

        while (j >= 0) {
            nums1[k] = nums2[j];
            j--;
            k--;
        }
    }
}
```

MinCoins.java

```
package old.Linkedin;

import org.testng.Assert;

public class MinCoins {
    // QUESTION: Given an array of coin denominations and a target value, find the
    // minimum number of coins
    // required to make up that value. You may assume that you have an infinite
    // supply of each coin type.
    // Example: Input: coins = [1,2,5,10], value = 13 -> Output: 3 (10 + 2 + 1)
    public static void main(String[] args) {
        int[] coinArray={1,2,5,10};
        Assert.assertEquals(minCoins(coinArray,13),3);
        Assert.assertEquals(minCoins(coinArray,20),2);
    }

    private static int minCoins(int[] coinArray, int value) {
        int count=0;
        for(int i=coinArray.length-1;i>=0;i--){
            while(value>=coinArray[i]){
                value-=coinArray[i];
                count++;
            }
        }
        return count;
    }
}
```

MinMaxArray.java

```
package old.Linkedin;

// QUESTION: Given an array of integers, find both the minimum and maximum
// elements in the array
// using an efficient approach that minimizes the number of comparisons.
// Example: Input: [12, 15, 7, 19, 3, 8] -> Output: Minimum: 3, Maximum: 19
public class MinMaxArray {
    public static void findMinMax(int[] arr) {
        if (arr == null || arr.length == 0) {
            System.out.println("Array is empty.");
            return;
        }
        int min, max;
        int i;
        int n = arr.length;

        if (n % 2 == 0) {
            if (arr[0] < arr[1]) {
```

```

        min = arr[0];
        max = arr[1];
    } else {
        min = arr[1];
        max = arr[0];
    }
    i = 2;
} else {
    min = max = arr[0];
    i = 1;
}

while (i < n - 1) {
    int num1 = arr[i];
    int num2 = arr[i + 1];

    if (num1 < num2) {
        min = Math.min(min, num1);
        max = Math.max(max, num2);
    } else {
        min = Math.min(min, num2);
        max = Math.max(max, num1);
    }
    i += 2;
}
System.out.println("Minimum: " + min);
System.out.println("Maximum: " + max);
}

public static void main(String[] args) {
    int[] arr = {12, 15, 7, 19, 3, 8};
    findMinMax(arr);
}
}

```

MoveNegativesTest.java

```

package old.Linkedin;

import org.testng.annotations.Test;
import static org.testng.Assert.assertEquals;

// QUESTION: Given an array of integers, move all negative numbers to the
// beginning of the array
// while maintaining the relative order among the positive numbers is not
// required.
// Example: Input: [1, 3, -1, -2, -3, 5, 6, -7] -> Output: [-1, -2, -3, -7, 1, 3,
// 5, 6]
public class MoveNegativesTest {

```

```

@Test
public void testMoveNegatives() {
    int[] nums1 = {1, 3, -1, -2, -3, 5, 6, -7};
    moveNegatives(nums1);
    assertEquals(nums1, new int[]{-1, -2, -3, -7, 1, 3, 5, 6}, "Test Case 1
Failed");

    int[] nums2 = {-5, -3, -1, -2, -4};
    moveNegatives(nums2);
    assertEquals(nums2, new int[]{-5, -3, -1, -2, -4}, "Test Case 2 Failed");

    int[] nums3 = {1, 2, 3, 4, 5};
    moveNegatives(nums3);
    assertEquals(nums3, new int[]{1, 2, 3, 4, 5}, "Test Case 3 Failed");

    int[] nums4 = {-1};
    moveNegatives(nums4);
    assertEquals(nums4, new int[]{-1}, "Test Case 4 Failed");

    int[] nums5 = {};
    moveNegatives(nums5);
    assertEquals(nums5, new int[]{}, "Test Case 5 Failed");

    int[] nums6 = {1};
    moveNegatives(nums6);
    assertEquals(nums6, new int[]{1}, "Test Case 6 Failed");

    int[] nums7 = {-1, 1, -2, 2, -3, 3};
    moveNegatives(nums7);
    assertEquals(nums7, new int[]{-1, -2, -3, 1, 2, 3}, "Test Case 7 Failed");
}

private void moveNegatives(int[] nums) {
    int j = 0;
    for (int i = 0; i < nums.length; i++) {
        if (nums[i] < 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            j++;
        }
    }
}
}

```

MoveNegativesWithOrderMaintain.java

```

package old.Linkedin;

import java.util.Arrays;

```

```

public class MoveNegativesWithOrderMaintain {
    // QUESTION: Given an array of integers, move all negative numbers to the
    // beginning of the array
    // while maintaining the relative order of both negative and positive numbers.
    // Example: Input: [-12, 11, -13, -5, 6, -7, 5, -3, -6]
    //              Output: [-12, -13, -5, -7, -3, -6, 11, 6, 5]

    public static void main(String[] args) {
        int[] arr = {-12, 11, -13, -5, 6, -7, 5, -3, -6}; // -12 -13 -5 -7 -3 -6 11
        moveNegatives(arr);
        System.out.println(Arrays.toString(arr));
    }

    private static void moveNegatives(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return;
        }
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] < 0) {
                if (i != j) {
                    int temp = nums[i];
                    for (int k = i; k > j; k--) {
                        nums[k] = nums[k - 1];
                    }
                    nums[j] = temp;
                }
                j++;
            }
        }
    }
}

```

NonRepChar.java

```

package old.Linkedin;

import java.util.HashMap;
import java.util.Map;

public class NonRepChar {
    // QUESTION: Given a string s, find the first non-repeating character in it
    // and return its index.
    // If it does not exist, return -1.
    // Example: Input: "geeksforgeeks" -> Output: 5 (character 'f' at index 5)

    public static void main(String[] args) {
        System.out.println(nonRepCharWithMap("geeksforgeeks"));
        System.out.println(nonRepChar("geeksforgeeks"));
    }
}

```



```

    }

    //Time Complexity: O(N)
    //Space Complexity: O(N)
    public static int nonRepCharWithMap(String str) {
        Map<Character, Integer> hm = new HashMap<>();
        for (Character c : str.toLowerCase().toCharArray()) {
            hm.put(c, hm.getOrDefault(c, 0) + 1);
        }
        for (int i = 0; i < str.length(); i++) {
            if (hm.get(str.charAt(i)) == 1) return i;
        }
        return -1;
    }

    //Time Complexity: O(N)
    //Space Complexity: O(1)
    public static int nonRepChar(String s) {
        int[] freq = new int[26];
        for (char c : s.toCharArray()) {
            freq[c - 'a']++;
        }
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }
        return -1;
    }
}

```

NthConsecutiveString.java

```

package old.Linkedin;

public class NthConsecutiveString {
    // QUESTION: Given a string, find the character that appears consecutively the
    // maximum number of times.
    // Example: Input: "Amazon is a great company as it has Atooozzzz" -> Output:
    // 'z'
    public static void main(String[] args) {
        String word = "Amazon is a great company as it has Atooozzzz";
        System.out.println(maxRepeating(word));
    }

    static char maxRepeating(String str) {
        int maxCount = 0;
        char result = str.charAt(0);

        int currentCount = 1;
        char currentChar = str.charAt(0);
    }
}

```

```

        for (int i = 1; i < str.length(); i++) {
            if (str.charAt(i) == currentChar) {
                currentCount++;
            } else {
                currentCount = 1;
                currentChar = str.charAt(i);
            }

            if (currentCount > maxCount) {
                maxCount = currentCount;
                result = currentChar;
            }
        }

        return result;
    }
}

```

NumberPalindrome.java

```

package old.Linkedin;

import java.util.Scanner;

public class NumberPalindrome {
    // QUESTION: Given an integer, determine if it is a palindrome. A palindrome
    // number reads the same
    // backward as forward. Negative numbers are not palindromes.
    // Example: Input: 121 -> Output: true
    //           Input: -121 -> Output: false

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        if (isPalindrome(number)) {
            System.out.println("Palindrome number");
        } else {
            System.out.println("Not a palindrome");
        }
    }

    static boolean isPalindrome(int x) {
        if (x < 0) return false;

        int temp = x;
        int num = 0;
        while (x != 0) {

```

```

        int rem = x % 10;
        num = num * 10 + rem;
        x /= 10;
    }
    return num == temp;
}
}

```

PairSumOfSortedArray.java

```

package old.Linkedin;

// QUESTION: Given a sorted array of integers and a target sum, find if there
// exists a pair of elements
// in the array that sum to the target. Return the pair if found.
// Example: Input: [1, 2, 3, 4, 5, 6], target = 8 -> Output: (2, 6) or (3, 5)
public class PairSumOfSortedArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};
        targetSum(arr, 8);
        targetSum(arr, 5);
    }

    static void targetSum(int[] arr, int targetSum) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int sum = arr[i] + arr[j];
            if (sum == targetSum) {
                System.out.println("Target Sum found: " + arr[i] + ", " + arr[j]);
                return;
            } else if (sum < targetSum) {
                i++;
            } else {
                j--;
            }
        }
        System.out.println("Target Sum not found.");
    }
}

```

Palindrome.java

```

package old.Linkedin;

public class Palindrome {
    // QUESTION: Given a string, determine if it is a palindrome. A palindrome
    // reads the same backward
}

```

```

// as forward. The comparison should be case-insensitive.
// Example: Input: "Madam" -> Output: true
//           Input: "Hello" -> Output: false

public static void main(String[] args) {
    String name = "Madam";
    StringBuilder rev = new StringBuilder();
    int count = name.length();

    // Reverse the string
    for (int i = count - 1; i >= 0; i--) {
        rev.append(name.charAt(i));
    }

    // Check if it is a palindrome (case-insensitive)
    if (name.equalsIgnoreCase(rev.toString())) {
        System.out.println("Palindrome");
    }
}
}

```

PermutationInString.java

```

package old.Linkedin;

import java.util.Arrays;

// QUESTION: Given two strings s1 and s2, return true if s2 contains a permutation
// of s1, or false otherwise.
// A permutation of a string is any rearrangement of its characters.
// Example: Input: s1 = "ab", s2 = "eidbaooo" -> Output: true (s2 contains "ba"
// which is a permutation of "ab")
public class PermutationInString {
    public static void main(String[] args) {
        String s1 = "ab", s2 = "eidbaooo";
        System.out.println(containsPermutations(s1, s2));
    }
    static boolean containsPermutations(String s1, String s2) {
        if (s1.length() > s2.length()) {
            return false;
        }

        int[] s1Freq = new int[26];
        int[] windowFreq = new int[26];

        for (char c : s1.toCharArray()) {
            s1Freq[c - 'a']++;
        }

        for (int i = 0; i < s2.length(); i++) {

```

```
        windowFreq[s2.charAt(i) - 'a']++;
        if (i >= s1.length()) {
            windowFreq[s2.charAt(i - s1.length()) - 'a']--;
        }
        if (Arrays.equals(s1Freq, windowFreq)) {
            return true;
        }
    }
    return false;
}
```

PrimeNumber.java

```
package old.Linkedin;

public class PrimeNumber {
    // QUESTION: Given an integer, determine if it is a prime number. A prime
    // number is a number greater
    // than 1 that has no positive divisors other than 1 and itself.
    // Example: Input: 23 -> Output: true
    //           Input: 4 -> Output: false

    public static void main(String[] args) {
        int num = 23;

        if (isPrime(num)) {
            System.out.println("Prime");
        } else {
            System.out.println("Not Prime");
        }
    }

    static boolean isPrime(int num) {
        if (num <= 1) {
            return false;
        }

        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                return false;
            }
        }

        return true;
    }
}
```

ProductArray.java

```
package old.Linkedin;

import java.util.Arrays;

// QUESTION: Given an integer array nums, return an array answer such that
// answer[i] is equal to the
// product of all the elements of nums except nums[i]. You must write an algorithm
// that runs in O(n)
// time and without using the division operation.
// Example: Input: [1, 2, 3, 4] -> Output: [24, 12, 8, 6]
public class ProductArray {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4};
        int[] result = productExceptSelf(nums);
        System.out.println(Arrays.toString(result));
    }

    public static int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        result[0] = 1;
        for (int i = 1; i < n; i++) {
            result[i] = result[i - 1] * nums[i - 1];
        }
        int rightProduct = 1;
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= rightProduct;
            rightProduct *= nums[i];
        }
        return result;
    }
}
```

RemoveDuplicates.java

```
package old.Linkedin;

import org.testng.Assert;

import java.util.Arrays;

// QUESTION: Given a sorted array of integers, remove the duplicates in-place such
// that each element
// appears only once. Return the array after removing duplicates.
// Example: Input: [1, 1, 2, 3, 4, 4, 4, 5] -> Output: [1, 2, 3, 4, 5]
public class RemoveDuplicates {
    public static void main(String[] args) {
```

```

        int[] arr = {1, 1, 2, 3, 4, 4, 4, 5};
        Assert.assertEquals(removeDuplicates(arr), new int[]{1, 2, 3, 4, 5});
    }

    public static int[] removeDuplicates(int[] arr) {
        if (arr == null || arr.length == 0) return new int[0];
        Arrays.sort(arr);
        int j = 0;

        for (int i = 1; i < arr.length; i++) {
            if (arr[i] != arr[j]) { //Shifts unique elements forward instead of
swapping
                j++;
                arr[j] = arr[i];
            }
        }
        return Arrays.copyOfRange(arr, 0, j + 1);
    }
}

```

RemoveStringDuplicates.java

```

package old.Linkedin;

// QUESTION: Given a string, remove duplicate characters from it while preserving
the order of
// first occurrence.
// Example: Input: "geeksforgeeks" -> Output: "geksfor"
public class RemoveStringDuplicates {
    public static String removeDuplicates(String str) {
        boolean[] visited = new boolean[256];
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (!visited[ch]) {
                visited[ch] = true;
                sb.append(ch);
            }
        }

        return sb.toString();
    }

    public static void main(String[] args) {
        String str = "geeksforgeeks";
        String result = removeDuplicates(str);
        System.out.println(result); // Output: geksfors
    }
}

```

Reverse.java

```
package old.Linkedin;

// QUESTION: Given a string, reverse the entire string and also reverse the order
// of words in the string.
// Example: Input: "My Name is Saurav Singh"
//           Reverse words: "Singh Saurav is Name My"
//           Reverse string: "hgniS varuaS si emaN yM"
public class Reverse {
    public static void main(String[] args) {
        String input = "My Name is Saurav Singh";

        // Reversing words
        String reversedWords = reverseWords(input);
        System.out.println("Reversed Words: " + reversedWords);

        // Reversing the entire string
        String reversedString = reverseString(input);
        System.out.println("Reversed String: " + reversedString);
    }

    static String reverseWords(String input) {
        String[] words = input.split(" ");
        StringBuilder reversed = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            reversed.append(words[i]).append(" ");
        }

        return reversed.toString().trim();
    }

    static String reverseString(String input) {
        char[] characters = input.toCharArray();
        StringBuilder reversed = new StringBuilder();

        for (int i = characters.length - 1; i >= 0; i--) {
            reversed.append(characters[i]);
        }

        return reversed.toString();
    }
}
```

ReverseArray.java

```
package old.Linkedin;
```



```
// QUESTION: Given an array of integers, reverse the array in-place.
// Example: Input: [1, 2, 3, 4, 5, 6] -> Output: [6, 5, 4, 3, 2, 1]
public class ReverseArray {
    public static void main(String[] args) {
        int[] array = {1, 2, 3, 4, 5, 6};
        int start = 0, end = array.length - 1;
        while (start < end) {
            int temp = array[start];
            array[start] = array[end];
            array[end] = temp;
            start++;
            end--;
        }
        for (int a : array) {
            System.out.print(a + " ");
        }
    }
}
```

ReverseString.java

```
package old.Linkedin;

public class ReverseString {
    // QUESTION: Given a string, reverse each word in the string while maintaining the
    // original order of words.
    // Example: Input: "India is my country" -> Output: "aidnI si ym yrtnuoc"
    public static void main(String[] args) {
        //Space/Time: O(n*m)
        String str="India is my country";
        String[] arr=str.split("\\s+");
        StringBuilder result=new StringBuilder();
        for(int i=arr.length-1;i>=0;i--){
            char[] wordArray=arr[i].toCharArray();
            for(int j=wordArray.length-1;j>=0;j--){
                result.append(wordArray[j]);
            }
            result.append(" ");
        }
        System.out.println(result);
    }
}
```

ReverseStringWithoutSpecialChars.java

```
package old.Linkedin;
```

```

public class ReverseStringWithoutSpecialChars {
    // QUESTION: Given a string containing alphabetic characters and special
    // characters, reverse only
    // the alphabetic characters while keeping the special characters in their
    // original positions.
    // Example: Input: "a!b?c" -> Output: "c?b!a"
    //Time Complexity: O(n)
    //Space Complexity: O(n)
    public static String reverseString(String str) {
        char[] charArray = str.toCharArray();
        int left = 0, right = charArray.length - 1;

        while (left < right) {
            if (!Character.isAlphabetic(charArray[left])) {
                left++;
            } else if (!Character.isAlphabetic(charArray[right])) {
                right--;
            } else {
                char temp = charArray[left];
                charArray[left] = charArray[right];
                charArray[right] = temp;
                left++;
                right--;
            }
        }

        return new String(charArray);
    }

    public static void main(String[] args) {
        String str = "a!b?c";
        String reversedStr = reverseString(str);
        System.out.println(reversedStr); // Output: c?b!a
    }
}

```

SecondLargest.java

```

package old.Linkedin;

public class SecondLargest {
    // QUESTION: Given an array of integers, find the second largest element in
    // the array.
    // If there is no second largest element (i.e., all elements are the same),
    // return an appropriate message.
    // Example: Input: [1, 2, 3, 1, 2, 3, 4] -> Output: 3
    //Time Complexity: O(n)
    //Space Complexity: O(1)
    public static void main(String[] args) {
        int[] arr = new int[]{1, 2, 3, 1, 2, 3, 4};
        int largest = Integer.MIN_VALUE;
    }
}

```

```

    int secondLargest = Integer.MIN_VALUE;

    for (int num : arr) {
        if (num > largest) {
            secondLargest = largest;
            largest = num;
        } else if (num > secondLargest && num != largest) {
            secondLargest = num;
        }
    }

    if (secondLargest == Integer.MIN_VALUE) {
        System.out.println("No second largest element found.");
    } else {
        System.out.println("Second largest element: " + secondLargest);
    }
}
}

```

SecondMostFrequentChar.java

```

package old.Linkedin;

public class SecondMostFrequentChar {
    // QUESTION: Given a string, find the second most frequent character in it.
    // Example: Input: "banana" -> Output: 'n' (b:1, a:3, n:2)
    public static void main(String[] args) {
        String name = "banana";
        char[] arr = name.toCharArray();
        int[] counter = new int[arr.length];

        for (int i = 0; i < arr.length; i++) {
            int loopCount = 1;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) {
                    loopCount++;
                    counter[j] = -1;
                }
            }
            if (counter[i] != -1) {
                counter[i] = loopCount;
            }
        }

        int largest = 0;
        int secLar = 0;

        for (int i = 0; i < arr.length; i++) {
            if (counter[i] > counter[largest]) {
                secLar = largest;
            }
        }
    }
}

```

```

        largest = i;
    } else if (counter[i] > counter[secLar] && counter[i] !=
counter[largest]) {
        secLar = i;
    }
}

System.out.println("Second most frequent character: " + arr[secLar]);
}
}

```

SecondMostFrequentString.java

```

package old.Linkedin;

import java.util.HashMap;
import java.util.Map;

// QUESTION: Given a string of space-separated words, find the second most
frequent word in the string.
// If there is no second most frequent word (i.e., all words have the same
frequency), return null.
// Example: Input: "apple banana apple orange banana apple banana grape banana
orange orange"
//          Output: "orange" (frequency 3, while "banana" has frequency 4 and
"apple" has frequency 3)
public class SecondMostFrequentString {
    public static void main(String[] args) {
        String input = "apple banana apple orange banana apple banana grape banana
orange orange";
        String[] words=input.split(" ");

        Map<String, Integer> frequencyMap = new HashMap<>();
        for (String word : words) {
            if (!word.isEmpty()) {
                frequencyMap.put(word, frequencyMap.getOrDefault(word, 0) + 1);
            }
        }
        frequencyMap.forEach((word, frequency) -> System.out.println(word + ": " +
frequency));
        String secondMostFrequent = getSecondMostFrequent(frequencyMap);
        System.out.println("Second most frequent string: " + secondMostFrequent);
    }

    private static String getSecondMostFrequent(Map<String, Integer> frequencyMap)
    {
        String secondMostFrequent = null;
        int secondMaxFrequency = 0;
        int maxFrequency = 0;
    }
}

```

```

        for (Map.Entry<String, Integer> entry : frequencyMap.entrySet()) {
            String word = entry.getKey();
            int frequency = entry.getValue();

            if (frequency > maxFrequency) {
                secondMaxFrequency = maxFrequency;
                secondMostFrequent = word;
                maxFrequency = frequency;
            } else if (frequency > secondMaxFrequency && frequency < maxFrequency)
            {
                secondMaxFrequency = frequency;
                secondMostFrequent = word;
            }
        }
        return secondMostFrequent;
    }
}

```

SecondSmallestLargest.java

```

package old.Linkedin;

public class SecondSmallestLargest {
    // QUESTION: Given an array of integers, find both the second smallest and
    // second largest elements.
    // Example: Input: [1, 8, 5, 3, 20, 4, 5, 8, 6, 10] -> Output: Second
    // Smallest: 3, Second Largest: 10
    public static void main(String[] args) {
        int[] arr = {1, 8, 5, 3, 20, 4, 5, 8, 6, 10};
        int secondSmallest = findSecondSmallest(arr);
        System.out.println("Second Smallest Element: " + secondSmallest);
        System.out.println("Second Largest Element: " +
            findSecondLargestElement(new int[]{1, 8, 5, 3, 20, 4, 5, 8, 6, 10}));
    }

    public static int findSecondLargestElement(int[] array) {
        if (array == null || array.length < 2) {
            return -1; // Array has fewer than two elements
        }

        int firstLargest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;

        for (int element : array) {
            if (element > firstLargest) {
                secondLargest = firstLargest;
                firstLargest = element;
            } else if (element > secondLargest && element != firstLargest) {
                secondLargest = element;
            }
        }
    }
}

```

```

    }

    return (secondLargest == Integer.MIN_VALUE) ? -1 : secondLargest;
}

static int findSecondSmallest(int[] arr) {
    int smallest = Integer.MAX_VALUE;
    int secondSmallest = Integer.MAX_VALUE;

    for (int j : arr) {
        if (j < smallest) {
            secondSmallest = smallest;
            smallest = j;
        } else if (j < secondSmallest && j != smallest) {
            secondSmallest = j;
        }
    }

    return secondSmallest;
}
}

```

SegrigateZeroAtLast.java

```

package old.Linkedin;

// QUESTION: Given an array of integers, move all zeros to the end of the array
// while maintaining
// the relative order of non-zero elements.
// Example: Input: [2, 3, 0, 0, 5, 0, 8] -> Output: [2, 3, 5, 8, 0, 0, 0]
public class SegrigateZeroAtLast {
    public static void main(String[] args) {
        int[] arr = {2, 3, 0, 0, 5, 0, 8};
        pushZerosToEnd(arr, arr.length);
        for (int j : arr) {
            System.out.print(j + " ");
        }
    }

    static void pushZerosToEnd(int[] arr, int n) {
        int count = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] != 0) {
                int temp = arr[count];
                arr[count] = arr[i];
                arr[i] = temp;
                count++;
            }
        }
    }
}

```

```
}
```

SeparateEvenOdd.java

```
package old.Linkedin;

import java.util.Arrays;

public class SeparateEvenOdd {
    // QUESTION: Given an array of integers, move all even numbers to the left
    // side and odd numbers to the right side.
    // Example: Input: [1, 2, 3, 4, 5, 6, 7, 8, 9] -> Output: [2, 4, 6, 8, 1, 3,
    // 5, 7, 9]
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6, 7, 8, 9};
        moveEven(arr);
        System.out.println(Arrays.toString(arr));
    }

    private static void moveEven(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return;
        }
        int j = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] % 2 == 0) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;
                j++;
            }
        }
    }
}
```

SeperateCaps.java

```
package old.Linkedin;

public class SeperateCaps {
    // QUESTION: Given a string containing lowercase letters, uppercase letters,
    // and digits,
    // print the lowercase characters, uppercase characters, and digits
    // separately.
    // Example: Input: "sumitM28" -> Output: sumit, M, 28

    public static void main(String[] args) {
```

```

        String input = "sumitM28";
        input.chars().filter(Character::isLowerCase).forEach(c ->
System.out.print((char) c));
        System.out.println();
        input.chars().filter(Character::isUpperCase).forEach(c ->
System.out.print((char) c));
        System.out.println();
        input.chars().filter(Character::isDigit).forEach(c ->
System.out.print((char) c));
        System.out.println();
    }

}

```

ShiftZeroes.java

```

package old.Linkedin;

import java.util.Arrays;

public class ShiftZeroes {
    // QUESTION: Given an array of integers, move all zeros to the end of the
    // array while maintaining
    // the relative order of non-zero elements.
    // Example: Input: [1, 4, 14, 0, 8, 9, 0, 5, 0, 6] -> Output: [1, 4, 14, 8, 9,
    // 5, 6, 0, 0, 0]
    //T:O(n)
    //S:O(1)
    public static void main(String[] args) {
        int[] array = {1, 4, 14, 0, 8, 9, 0, 5, 0, 6};

        int[] resultFront = shiftZeroesToTheEnd(array);
        System.out.println("Zeroes to End: " + Arrays.toString(resultFront));
    }

    public static int[] shiftZeroesToTheEnd(int[] arr) {
        int count=0;
        for(int i=0;i<arr.length;i++){
            if(arr[i]!=0){
                int temp=arr[i];
                arr[i]=arr[count];
                arr[count]=temp;
                count++;
            }
        }
        return arr;
    }
}

```


SortCharArray.java

```
package old.Linkedin;

public class SortCharArray {
    // QUESTION: Given a string, sort its characters in ascending order.
    // Example: Input: "saurav" -> Output: "aarsuv"
    public static void main(String[] args) {
        String input = "saurav";
        char[] arr = input.toCharArray();

        // Sorting using bubble sort
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap arr[j] and arr[j+1]
                    char temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        // Printing the sorted string
        for (char c : arr) {
            System.out.print(c);
        }
    }
}
```

SortedArrayListEquality.java

```
package old.Linkedin;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;

public class SortedArrayListEquality {
    // QUESTION: Given two ArrayLists of integers, determine if they contain the
    // same elements
    // regardless of their initial order. Sort both lists and compare them.
    // Example: Input: [5, 2, 7, 1, 4] and [1, 2, 4, 5, 7] -> Output: true
    public static void main(String[] args) {
```

```

// Example 1: Equal ArrayLists
ArrayList<Integer> list1 = new ArrayList<>(Arrays.asList(5, 2, 7, 1, 4));
ArrayList<Integer> list2 = new ArrayList<>(Arrays.asList(1, 2, 4, 5, 7));

// Step 1: Sort both ArrayLists
Collections.sort(list1);
Collections.sort(list2);

// Step 2: Check for equality
boolean isEqual1 = list1.equals(list2);
System.out.println("Are sorted ArrayLists equal? " + isEqual1);

// Example 2: Different ArrayLists
ArrayList<Integer> list3 = new ArrayList<>(Arrays.asList(5, 2, 7, 1, 4));
ArrayList<Integer> list4 = new ArrayList<>(Arrays.asList(5, 4, 3, 2, 1));

// Step 1: Sort both ArrayLists
Collections.sort(list3);
Collections.sort(list4);

// Step 2: Check for equality
boolean isEqual2 = list3.equals(list4);
System.out.println("Are sorted ArrayLists equal? " + isEqual2);
}
}

```

SortedSquares.java

```

package old.Linkedin;

import java.util.Arrays;

public class SortedSquares {
    // QUESTION: Given a sorted array of integers (may contain negative numbers),
    // return an array
    // of the squares of each number sorted in non-decreasing order.
    // Example: Input: [-7, -3, 2, 3, 11] -> Output: [4, 9, 9, 49, 121]
    //t:s:n
    public static void main(String[] args) {
        int[] nums = {-7, -3, 2, 3, 11};
        int[] result = sortedSquares(nums);
        System.out.println(Arrays.toString(result));
    }

    public static int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];
        int left = 0;
        int right = n - 1;
        int index = n - 1;
    }
}

```

```

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];
            if (leftSquare > rightSquare) {
                result[index--] = leftSquare;
                left++;
            } else {
                result[index--] = rightSquare;
                right--;
            }
        }
        return result;
    }
}

```

SortHashMapByValue.java

```

package old.Linkedin;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Map.Entry;

public class SortHashMapByValue {
    // QUESTION: Given a HashMap, sort it by its values in both ascending and
    // descending order.
    // Example: Input: {apple=10, banana=5, cherry=15, date=2, elderberry=10}
    //           Output (ascending): {date=2, banana=5, apple=10, elderberry=10,
    //                               cherry=15}

    public static void main(String[] args) {
        HashMap<String, Integer> unsortedMap = new HashMap<>();
        unsortedMap.put("apple", 10);
        unsortedMap.put("banana", 5);
        unsortedMap.put("cherry", 15);
        unsortedMap.put("date", 2);
        unsortedMap.put("elderberry", 10);

        System.out.println("Unsorted HashMap: " + unsortedMap);

        Map<String, Integer> sortedMapAscending = sortByValue(unsortedMap);
        System.out.println("Sorted HashMap by value (ascending): " +
            sortedMapAscending);

        Map<String, Integer> sortedMapDescending =
            sortByValueDescending(unsortedMap);
    }
}

```

```

        System.out.println("Sorted HashMap by value (descending): " +
sortedMapDescending);
    }

    public static <K, V extends Comparable<? super V>> Map<K, V>
sortByValue(Map<K, V> map) {
        List<Entry<K, V>> list = new ArrayList<>(map.entrySet());
        list.sort(Entry.comparingByValue());
        Map<K, V> sortedMap = new java.util.LinkedHashMap<>();
        for (Entry<K, V> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }
        return sortedMap;
    }

    public static <K, V extends Comparable<? super V>> Map<K, V>
sortByValueDescending(Map<K, V> map) {
        List<Entry<K, V>> list = new ArrayList<>(map.entrySet());
        list.sort(Entry.<K, V>comparingByValue().reversed());
        Map<K, V> sortedMap = new java.util.LinkedHashMap<>();
        for (Entry<K, V> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }
        return sortedMap;
    }
}

```

SortMapByValue.java

```

package old.Linkedin;

import java.util.*;

public class SortMapByValue {
    // QUESTION: Given a Map, sort it by its values in descending order.
    // Example: Input: {apple=10, banana=5, orange=15, grape=8}
    //           Output: {orange=15, apple=10, grape=8, banana=5}

    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();
        map.put("apple", 10);
        map.put("banana", 5);
        map.put("orange", 15);
        map.put("grape", 8);

        List<Map.Entry<String, Integer>> list = new ArrayList<>
(map.entrySet());

        list.sort((o1, o2) -> o2.getValue().compareTo(o1.getValue()));

        Map<String, Integer> sortedMap = new TreeMap<>();
    }
}

```

```

        for (Map.Entry<String, Integer> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }
        System.out.println("Sorted Map by Value (Descending):");
        for (Map.Entry<String, Integer> entry : sortedMap.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}

```

SortStringCharsTest.java

```

package old.Linkedin;

import org.testng.Assert;
import org.testng.annotations.Test;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

public class SortStringCharsTest {
    // QUESTION: Given a string, sort its characters based on their frequency in
    // descending order.
    // If characters have the same frequency, maintain their original order.
    // Example: Input: "Aabb" -> Output: "bbAa" (b:2, A:1, a:1)
    public static String sortStrings(String s) {
        int[] freq = new int[256]; // Covers all ASCII characters
        for (char c : s.toCharArray()) {
            freq[c]++;
        }
        // Step 2: Create an array of characters for sorting
        Character[] chars = new Character[256];
        for (int i = 0; i < 256; i++) {
            chars[i] = (char) i;
        }
        // Step 3: Sort characters based on frequency in descending order
        Arrays.sort(chars, (a, b) -> freq[b] - freq[a]);

        // Step 4: Build the result string
        StringBuilder result = new StringBuilder();
        for (char c : chars) {
            if (freq[c] > 0) {
                result.append(String.valueOf(c).repeat(freq[c]));
            }
        }
        return result.toString();
    }
}

```

```

//t:nlogn,s:n
public static String sortStringsWithMaps(String s) {
    Map<Character, Integer> frequencyMap = new HashMap<>();

    for (Character c : s.toCharArray()) {
        frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
    }

    PriorityQueue<Map.Entry<Character, Integer>> pq = new PriorityQueue<>(
        (a, b) -> b.getValue() - a.getValue()
    );
    pq.addAll(frequencyMap.entrySet());

    StringBuilder sb = new StringBuilder();
    while (!pq.isEmpty()) {
        Map.Entry<Character, Integer> entry = pq.poll();
        char c = entry.getKey();
        int freq = entry.getValue();
        sb.append(String.valueOf(c).repeat(freq));
    }

    return sb.toString();
}

@Test
public void test() {
    Assert.assertEquals(sortStrings("Aabb"), "bbAa");
    Assert.assertEquals(sortStringsWithMaps("Aabb"), "bbAa");
}
}

```

Startingwith1.java

```

package old.Linkedin;

import java.util.Arrays;

public class Startingwith1 {
    // QUESTION: Given an array of integers, print all numbers that start with the
    // digit 1.
    // Example: Input: [1, 23, 123, 45, 134] -> Output: 1, 23, 123, 134
    public static void main(String[] args) {
        int[] arr = {1, 23, 123, 45, 134};
        Arrays.stream(arr).filter(num ->
            String.valueOf(num).startsWith("1")).forEach(System.out::println);
    }
}

```

StockBuySell.java

```
package old.Linkedin;

public class StockBuySell {
    // QUESTION: Given an array of stock prices where prices[i] is the price on
    // day i, find the
    // maximum profit you can achieve by buying on one day and selling on a
    // different day.
    // Example: Input: [7, 1, 5, 3, 6, 4] -> Output: 5 (buy at 1, sell at 6)
    public static void main(String[] args) {
        int[] prices = {7, 1, 5, 3, 6, 4};
        System.out.println(buyAndSellStocks(prices));
    }

    public static int buyAndSellStocks(int[] prices) {
        int buyPrice = Integer.MAX_VALUE;
        int maxProfit = 0;
        for (int price : prices) {
            if (price > buyPrice) {
                int profit = price - buyPrice;
                maxProfit = Math.max(maxProfit, profit);
            } else {
                buyPrice = price;
            }
        }
        return maxProfit;
    }
}
```

StringCompression.java

```
package old.Linkedin;

public class StringCompression {
    // QUESTION: Given a string, compress it by replacing consecutive repeating
    // characters with
    // the character followed by its count. If a character appears only once,
    // don't add a count.
    // Example: Input: "aaabbbacfwj" -> Output: "a3b3acfw3"
    //Time Complexity: O(n)
    //Space Complexity: O(n)

    public static void main(String[] args) {
        {
            String str = "aaabbbacfwj";
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < str.length(); i++) {
                int count = 1;
                while (i < str.length() - 1 && str.charAt(i) == str.charAt(i + 1))
```

```

{
    count++;
    i++;
}
sb.append(str.charAt(i));
if (count > 1) {
    sb.append(count);
}
}
System.out.println(sb);
}
}
}

```

StringPalindrome.java

```

package old.Linkedin;

public class StringPalindrome {
    // QUESTION: Given a string, determine if it is a palindrome. The comparison
    // should be case-insensitive.
    // Example: Input: "Nitin" -> Output: true
    //Time Complexity: O(n)
    //Space Complexity: O(n)
    public static void main(String[] args) {
        String name="Nitin";
        StringBuilder sb=new StringBuilder();
        for(int i=name.length()-1;i>=0;i--){
            sb.append(name.toLowerCase().charAt(i));
        }
        boolean ans=name.toLowerCase().contentEquals(sb);
        System.out.println(ans);
    }
}

```

StringPermutations.java

```

package old.Linkedin;

import java.util.HashSet;
import java.util.Set;

public class StringPermutations {
    // QUESTION: Given a string, generate all possible permutations of its
    // characters.
    // Example: Input: "geek" -> Output: All permutations of "geek"
    //t:n!,s:n

```



```

public static void main(String[] args) {
    //printPermutn("geek", "");
    System.out.println();
    System.out.print(permute("geek"));
}

static void printPermutn(String str, String ans) {
    if (str.isEmpty()) {
        System.out.print(ans + " ");
        return;
    }

    boolean[] alpha = new boolean[26];
    for (int i = 0; i < str.length(); i++) {
        char ch = str.charAt(i);
        String ros = str.substring(0, i) + str.substring(i + 1);
        if (!alpha[ch - 'a']) printPermutn(ros, ans + ch);
        alpha[ch - 'a'] = true;
    }
}

static Set<String> permute(String str) {
    if (str == null) {
        return null;
    } else if (str.isEmpty()) {
        Set<String> set = new HashSet<>();
        set.add("");
        return set;
    }
    char firstChar = str.charAt(0);
    String remainingStr = str.substring(1);
    Set<String> words = permute(remainingStr);
    Set<String> permutations = new HashSet<>();
    for (String word : words) {
        for (int j = 0; j <= word.length(); j++) {
            String s = word.substring(0, j) + firstChar + word.substring(j);
            permutations.add(s);
        }
    }
    return permutations;
}
}

```

SwapWith.java

```

package old.Linkedin;

import java.util.Scanner;

public class SwapWith {

```

```
// QUESTION: Swap two numbers using a temporary variable and also without
// using a temporary variable.
// Example: Input: x=10, y=5 -> Output: x=5, y=10
public static void main(String[] args) {
    int x, y, t; // x and y are to swap
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the value of X and Y");
    x = sc.nextInt();
    y = sc.nextInt();
    System.out.println("before swapping numbers: " + x + " " + y);
    /* swapping */
    t = x;
    x = y;
    y = t;
    System.out.println("After swapping: " + x + " " + y);
    System.out.println();

    int x1 = 10;
    int y1 = 5;
    x1 = x1 + y1;
    y1 = x1 - y1;
    x1 = x1 - y1;
    System.out.println("After swapping: x = " + x1 + ", y = " + y1);
}
}
```

TrappedRainWater.java

```
package old.Linkedin;

public class TrappedRainWater {
    // QUESTION: Given an array of non-negative integers representing the height
    // of bars, compute how
    // much water it can trap after raining.
    // Example: Input: [4,2,0,6,3,2,5] -> Output: 9
    public static void main(String[] args) {
        int[] height=new int[]{4,2,0,6,3,2,5};
        int waterTrapped=trapRainWater(height);
        System.out.println(waterTrapped);
    }

    public static int trapRainWater(int[] height){
        int length=height.length;
        int[] leftMaxBoundary=new int[length];
        int[] rightMaxBoundary=new int[length];

        leftMaxBoundary[0]=height[0];
        for(int i=1;i<length;i++){
            leftMaxBoundary[i]=Math.max(leftMaxBoundary[i-1],height[i]);
        }
    }
}
```

```

    }

    rightMaxBoundary[length-1]=height[length-1];
    for(int i=length-2;i>=0;i--){
        rightMaxBoundary[i]=Math.max(height[i],rightMaxBoundary[i+1]);
    }
    int result=0;
    for(int i=0;i<length;i++){
        int waterLevel=Math.min(leftMaxBoundary[i],rightMaxBoundary[i]);
        result+=waterLevel-height[i];
    }

    return result;
}
}

```

TrianglePattern.java

```

package old.Linkedin;

public class TrianglePattern {
    // QUESTION: Print a right-angled triangle pattern of stars with the given
    // number of rows.
    // Example: Input: n = 5
    //           Output:
    //           *
    //          * *
    //         * * *
    //        * * * *
    //       * * * * *

    // For Outer loop use Y axis(column)
    // For inner Loop use X Axis(row)
    // https://1drv.ms/b/s!AvMeJE10kAhtgaMdDCM6W10j08R8XQ?e=iKY0rt

    public static void printRightAngledTriangle(int n) {
        for (int i = 0; i < n; i++) {
            for (int j = n - i; j > 1; j--) {
                System.out.print(" ");
            }
            for (int j = 0; j <= i; j++) {
                System.out.print("* ");
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        int n = 5;
        printRightAngledTriangle(n);
    }
}

```

```

    }
}

```

ValidBrackets.java

```

package old.Linkedin;

import java.util.Stack;

public class ValidBrackets {
    // QUESTION: Given a string containing just the characters '(', ')', '{', '}',
    // '[' and ']',
    // determine if the input string is valid. An input string is valid if open
    // brackets are closed
    // by the same type of brackets and in the correct order.
    // Example: Input: "()[]{}" -> Output: true
    //           Input: "([)]" -> Output: true
    // Space/Time Complexity: O(n)

    public static void main(String[] args) {
        String input = "()[]{}";
        System.out.println(isValid(input)); // Output: true

        input = "([)]";
        System.out.println(isValid(input)); // Output: true
    }

    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();

        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else if (c == ')' && !stack.isEmpty() && stack.peek() == '(') {
                stack.pop();
            } else if (c == '}' && !stack.isEmpty() && stack.peek() == '{') {
                stack.pop();
            } else if (c == ']' && !stack.isEmpty() && stack.peek() == '[') {
                stack.pop();
            } else {
                return false;
            }
        }
        return stack.isEmpty();
    }
}

```

ZeroSumSubarrays.java

```
package old.Linkedin;

import java.util.*;

public class ZeroSumSubarrays {
    // QUESTION: Given an array of integers, find all subarrays that sum to zero.
    // Print each subarray.
    // Example: Input: [6, 3, -1, -3, 4, -2, 2, 4, 6, -12, -7]
    //             Output: All subarrays that sum to zero
    public static void printZeroSumSubarrays(int[] arr) {
        Map<Integer, List<Integer>> map = new HashMap<>();
        map.put(0, new ArrayList<>(List.of(-1))); // To handle cases where the
        subarray starts from index 0

        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
            if (map.containsKey(sum)) {
                List<Integer> indices = map.get(sum);
                for (int startIdx : indices) {
                    System.out.println("Subarray found: " +
                        Arrays.toString(Arrays.copyOfRange(arr, startIdx + 1, i + 1)));
                }
            }
            map.putIfAbsent(sum, new ArrayList<>());
            map.get(sum).add(i);
        }
    }

    public static void main(String[] args) {
        int[] arr = {6, 3, -1, -3, 4, -2, 2, 4, 6, -12, -7};
        System.out.println("Zero-sum subarrays:");
        printZeroSumSubarrays(arr);
    }
}
```

Topic: Notion

Code Solutions

AppendFile.java

```
package old.Notion;

import java.io.*;

public class AppendFile {
```

```

public static void main(String[] args) {
    String text = "Welcome to GeeksforGeeks\nHappy Learning!";
    String filePath = "src/test/resources/file3.txt";

    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath,
true))) {
        writer.write(text);
        System.out.println("Successfully appended to the file: " + filePath);
    } catch (IOException e) {
        System.err.println("An error occurred while appending to the file.");
        e.printStackTrace();
    }

    System.out.println("\n--- Reading file contents ---");

    try (BufferedReader reader = new BufferedReader(new FileReader(filePath)))
    {
        String currentLine;
        while ((currentLine = reader.readLine()) != null) {
            System.out.println(currentLine);
        }
    } catch (IOException e) {
        System.err.println("An error occurred while reading the file.");
        e.printStackTrace();
    }
}

```

CompareFiles.java

```

package old.Notion;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Iterator;
import java.util.stream.Stream;

public class CompareFiles {
    public static void main(String[] args) {
        String file1 = "src/test/resources/file1.txt";
        String file2 = "src/test/resources/file2.txt";
        try (
            Stream<String> fileStream1 = Files.lines(Paths.get(file1));
            Stream<String> fileStream2 = Files.lines(Paths.get(file2))
        ) {
            Iterator<String> it1 = fileStream1.iterator();
            Iterator<String> it2 = fileStream2.iterator();
            int lineNum = 1;
            while (it1.hasNext() || it2.hasNext()) {
                String line1 = it1.hasNext() ? it1.next() : "<no line>";

```

```

        String line2 = it2.hasNext() ? it2.next() : "<no line>";
        if (!line1.equals(line2)) {
            System.out.printf("Difference at line %d:\nFile1: %s\nFile2:
%s\n\n", lineNum, line1, line2);
        }
        lineNum++;
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}

```

CountLogs.java

```

package old.Notion;

import org.testng.annotations.Test;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class CountLogs {

    @Test
    public void countLogLevelsBufferedTest() {
        Path path = Paths.get("src/test/resources/application.log");
        Map<String, Integer> logLevels = countLogLevelsBuffered(path);
        System.out.println(logLevels);
    }

    @Test
    public void countLogLevelsTest() {
        Path path = Paths.get("src/test/resources/application.log");
        Map<String, Long> logLevels = countLogLevels(path);
        System.out.println(logLevels);
    }

    @Test
    public void findLongestLineTest() {
        Path path = Paths.get("src/test/resources/emp.txt");
    }
}

```

```
        findLongestLine(path).ifPresent(line -> {
            System.out.println("Longest Line:");
            System.out.println(line);

            System.out.println("\nLength:");
            System.out.println(line.length());
        });
    }

    public static Map<String, Long> countLogLevels(Path path) {
        try (Stream<String> lines = Files.lines(path)) {
            return lines
                .map(String::trim)
                .filter(line -> !line.isEmpty())
                .map(line -> line.split("\\s+")[2])
                .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()));
        } catch (IOException e) {
            throw new RuntimeException("Failed to read log file", e);
        }
    }

    public static Optional<String> findLongestLine(Path path) {
        try (Stream<String> lines = Files.lines(path)) {
            return lines.max(Comparator.comparingInt(String::length));
        } catch (IOException e) {
            throw new RuntimeException("Failed to read lines for longest line
check", e);
        }
    }

    public static Map<String, Integer> countLogLevelsBuffered(Path path) {
        Map<String, Integer> counts = new HashMap<>();
        try (BufferedReader reader = Files.newBufferedReader(path)) {
            String line;
            while ((line = reader.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) continue;
                String level = line.split("\\s+")[2];
                counts.merge(level, 1, Integer::sum);
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
        return counts;
    }
}
```



```
package old.Notion;

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.stream.Stream;

public class FileIO {
    public static void main(String[] args) {
        readWriteWithBuffer();
        readWriteWithStream();
    }

    private static void readWriteWithBuffer() {
        String filePath = "src/test/resources/file3.txt";

        try (BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(new
FileOutputStream(filePath, true), StandardCharsets.UTF_8))) {
            writer.write("Hello World");
            writer.newLine();
            writer.write(String.valueOf(Math.random()));
            writer.newLine();
        } catch (Exception e) {
            System.err.println("Error writing to file: " + e.getMessage());
            e.printStackTrace();
        }

        try (BufferedReader reader = new BufferedReader(new InputStreamReader(new
FileInputStream(filePath), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (Exception e) {
            System.err.println("Error reading from file: " + e.getMessage());
            e.printStackTrace();
        }
    }

    private static void readWriteWithStream() {
        Path path = Paths.get("src/test/resources/file3.txt");

        try {
            Files.writeString(path, "Hello World\n" + Math.random() + "\n",
StandardCharsets.UTF_8, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
        } catch (IOException e) {
            System.err.println("Error writing to file: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

```
try (Stream<String> lines = Files.lines(path, StandardCharsets.UTF_8)) {
    lines.forEach(System.out::println);
} catch (IOException e) {
    System.err.println("Error reading from file: " + e.getMessage());
    e.printStackTrace();
}
}
```

LL.java

```
package old.Notion;

public class LL {
    static class Node {
        int value;
        Node next;

        public Node(int value) {
            this.value = value;
        }
    }

    static class SinglyLinkedList {
        Node head, tail;

        public SinglyLinkedList() {
            head = null;
            tail = null;
        }

        public void insertAtHead(int value) {
            Node node = new Node(value);
            if (head == null) {
                head = tail = node;
                return;
            }
            node.next = head;
            head = node;
        }

        public void insertAtTail(int value) {
            Node node = new Node(value);
            if (head == null) {
                head = tail = node;
                return;
            }
            tail.next = node;
            tail = node;
        }
    }
}
```

```
}

public void deleteValue(int value) {
    if (head == null) return;

    if (head.value == value) {
        if (head == tail) tail = null;
        head = head.next;
        return;
    }

    Node p1 = head;
    Node p2 = null;

    while (p1 != null) {
        if (p1.value == value) {
            if (p1 == tail) tail = p2;
            p2.next = p1.next;
            return;
        }
        p2 = p1;
        p1 = p1.next;
    }
}

public void print() {
    Node p1 = head;
    while (p1 != null) {
        System.out.print(p1.value + " -> ");
        p1 = p1.next;
    }
    System.out.println("null");
}

}

public static void main(String[] args) {
    SinglyLinkedList list = new SinglyLinkedList();
    list.insertAtHead(0);
    list.insertAtTail(1);
    list.insertAtTail(2);
    list.insertAtTail(3);

    System.out.println("Initial list:");
    list.print();

    System.out.println("Deleting 2:");
    list.deleteValue(2);
    list.print();

    System.out.println("Deleting 0 (Head):");
    list.deleteValue(0);
    list.print();
}
```

```
}
}
```

PasswordSanitizer.java

```
package old.Notion;

import java.util.ArrayList;
import java.util.List;

class PasswordSanitizer {
    // Filter passwords with condition that, password must have atleast 5
    // characters and should be alphanumeric
    public static String filter(List<String> passwords) {
        List<String> validPass = new ArrayList<>();

        for (String password : passwords) { // <-- FIXED this line
            boolean isLongEnough = password.length() >= 5;
            boolean containsOnlyLet = password.matches("[a-zA-Z]+");
            boolean containsOnlyNum = password.matches("[0-9]+");
            boolean isMixed = !(containsOnlyLet || containsOnlyNum);

            if (isLongEnough && isMixed) {
                validPass.add(password);
            }
        }

        return String.join(" ", validPass);
    }

    public static void main(String[] args) {
        List<String> passwords = new ArrayList<>();
        passwords.add("peC1");
        passwords.add("pass@123");
        passwords.add("word@321");
        passwords.add("wordpass");
        passwords.add("987345");
        System.out.println(filter(passwords));
    }
}
```

ReadLogs.java

```
package old.Notion;

import java.io.*;
```

```
import java.nio.file.Paths;

public class ReadLogs {
    public static void main(String[] args) {
        String inputFile = "src/test/resources/application.log";
        String outputFile = "src/test/resources/error.log";
        try (BufferedReader reader = new BufferedReader(new
        FileReader(inputFile));
            BufferedWriter writer = new BufferedWriter(new
        FileWriter(outputFile))) {
            String currentLine;
            while ((currentLine = reader.readLine()) != null) {
                if (currentLine.contains("ERROR")) {
                    writer.write(currentLine);
                    writer.newLine();
                }
            }
            System.out.println("Successfully extracted error logs to: " +
        Paths.get(outputFile).toAbsolutePath());
        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

ReadLogsModern.java

```
package old.Notion;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.List;
import java.util.stream.Collectors;

public class ReadLogsModern {
    public static void main(String[] args) {
        Path inputFile = Paths.get("src/test/resources/application.log");
        Path outputFile = Paths.get("src/test/resources/error.log");
        try {
            List<String> errorLines = Files.lines(inputFile).filter(line ->
        line.contains("ERROR")).collect(Collectors.toList());
            Files.write(outputFile, errorLines, StandardOpenOption.CREATE,
        StandardOpenOption.APPEND);
            System.out.println("Successfully extracted error logs to: " +
        outputFile.toAbsolutePath());
        } catch (IOException e) {
            System.err.println("An I/O error occurred: " + e.getMessage());
        }
    }
}
```

```
        e.printStackTrace();  
    }  
}  

```
